



**Tecnológico
de Monterrey**

**MA2003B Aplicación de Métodos Multivariados
en Ciencia de Datos**

Módulo 1: Relaciones de Interdependencia

Profesor: Raul Gomez
Correo: rgomez@tec.mx
Oficina: A4-123A

Tabla de contenidos

I	Pruebas de Normalidad Univariada	3
1.	Introducción al Tidyverse	5
1.1.	Datos ordenados	5
1.1.1.	Tibbles y pipes	5
1.1.2.	Ejemplo: Ordenando el conjunto de datos de la OMS	6
1.2.	La “Gramática de los Gráficos” y ggplot2	9
1.2.1.	Capas en ggplot2	9
1.2.2.	Graficación con ggplot2	10
1.3.	Actividad: Limpieza de datos	11
2.	Los momentos estandarizados	15
2.1.	Asimetría	15
2.2.	Curtosis	18
2.3.	Gráficos QQ	20
2.4.	Asimetría y curtosis muestrales	24
2.5.	Escalamiento y estandarización	26
2.5.1.	Escalamiento de variables aleatorias	26
2.5.2.	Escalamiento de muestras	29
2.6.	Actividad: Visualizando la no normalidad de muestras	34
3.	Pruebas de normalidad	35
3.1.	Pruebas de normalidad	35
3.2.	Transformaciones	36
3.3.	Optimización de normalidad (Box–Cox)	41
II	Análisis de Componentes Principales	45
4.	Introducción al análisis multivariante	47
4.1.	El vector de medias y la matriz de covarianzas	47
4.2.	La distancia de Mahalanobis	49
4.3.	Escalamiento y estandarización	55
4.4.	Actividad: La geometría de los datos multivariantes	55
5.	Distribución normal multivariante	57

5.1. Definición y propiedades básicas	57
5.2. Contornos de la normal multivariante	58
5.3. Estandarización de la normal multivariante	61
5.4. Interpretación Geométrica de la SVD	61
5.5. Clausura bajo transformaciones lineales	64
5.6. Pruebas de normalidad multivariante	65
6. Análisis de componentes principales	67
6.1. Entendiendo la varianza total	67
6.2. Varianza acumulada y gráficos scree	72
6.3. Cargas (loadings) y biplots	76
6.4. Resumen del proceso de PCA	80
 III Análisis Factorial	 83
7. Análisis factorial: modelos teóricos	85
7.1. Planteamiento del análisis factorial	85
7.2. Descomposición de varianza	86
7.3. Estimación de puntuaciones factoriales	87
7.4. Rotación de factores	87
8. Análisis factorial: métodos prácticos	89
8.1. Evaluación de la factorizabilidad	89
8.2. Decidir el número de factores	90
8.3. Extracción de factores	90
8.4. Rotación de factores	90
8.5. Cálculo de puntuaciones factoriales	91
8.6. Evaluación del ajuste global	91
8.7. Visualización del modelo	91
8.8. Ejemplo: análisis factorial del conjunto Holzinger-Swineford	92
8.8.1. Evaluar la factorizabilidad	92
8.8.2. Decidir el número de factores	94
8.8.3. Extracción, rotación y puntuación de factores	96
8.8.4. Evaluación del ajuste del modelo	97
8.8.5. Visualización del modelo	98
 IV Análisis de Conglomerados	 103
9. Análisis de conglomerados jerárquicos	105
9.1. Entendiendo los dendrogramas	105
9.2. Realización del agrupamiento jerárquico	111
10. Agrupamiento no jerárquico	113
10.1. Métodos de particionado	113
10.2. Métodos basados en densidad	113

10.3. Métodos basados en modelos	114
10.4. Elección del método adecuado	114
10.5. Evaluación de los resultados de agrupamiento	115
10.6. Ejemplo en R	115

Parte I

Pruebas de Normalidad Univariada

Capítulo 1

Introducción al Tidyverse

El Tidyverse es una colección de paquetes de R diseñados para la ciencia de datos. Proporciona una interfaz coherente y amigable para la manipulación, visualización y análisis de datos. Aunque se introdujo formalmente como ecosistema en 2016, muchos de estos paquetes fueron desarrollados por Hadley Wickham entre 2007 y 2014.

1.1. Datos ordenados

La idea central detrás del diseño del Tidyverse es la definición de “datos ordenados” de Wickham [3].

Definición 1.1.1. *Datos ordenados* es una forma de estructurar conjuntos de datos para facilitar su uso. En datos ordenados:

1. Cada variable forma una columna.
2. Cada observación forma una fila.
3. Cada tipo de unidad observacional forma una tabla.

Esta estructura facilita la manipulación, el análisis y la visualización de datos, ya que se alinea con la forma en que los datos se procesan típicamente en R. En esta sección describiremos los conceptos básicos del Tidyverse; para más información sobre su uso, el lector interesado puede consultar: [5, 6, 7].

1.1.1. Tibbles y pipes

En el Tidyverse, los datos suelen representarse mediante un tipo especial de data frame llamado *tibble*. Los tibbles son más amigables que los data frames tradicionales, pues ofrecen mejor impresión y comportamiento de subconjuntos. Además, están diseñados para trabajar de forma fluida con el operador pipe (`%>%`), que permite encadenar múltiples operaciones (conocidas como “verbos” en el contexto del Tidyverse) de manera clara y concisa. Para ilustrar estas ideas, en la siguiente sección realizaremos la limpieza del conjunto de datos de la OMS (“WHO”), para asegurar que el tibble asociado cumpla los principios de datos ordenados. Esto suele ser el primer paso del

Análisis Exploratorio de Datos (EDA) y es crucial para garantizar que los datos estén en un formato adecuado para su análisis y visualización.

1.1.2. Ejemplo: Ordenando el conjunto de datos de la OMS

Antes de comenzar, debemos cargar las librerías necesarias y el conjunto de datos de la OMS:

```
library("tidyverse")
library("here")
library("cowplot")
library("patchwork")
library("krulRutils")
library("ISLR2")
library("magrittr")

options(scipen = 999) # Desactivar notación científica
data(who)
```

Ahora podemos iniciar nuestro proceso de “ordenamiento”. El principal problema de este conjunto es que contiene variables (como “new_sp_m014”) que no son muy claras y que incluyen más de una pieza de información. Por ello necesitamos separar estas variables e introducir mejores nombres para ellas y sus valores asociados.

```
# Empezamos creando la variable "who_tidy"
# que contiene el conjunto de datos de la OMS ya limpio
who_tidy <- who %>%
  # Usamos pivot_longer para eliminar las variables que empiezan con "new"
  # y usarlas como valores en su lugar
  pivot_longer(
    cols = starts_with("new"),
    names_to = "key",
    values_to = "cases",
    values_drop_na = TRUE
  ) %>%
  mutate(
    key = if_else(
      startsWith(key, "newrel"),
      sub("newrel", "new_rel", key),
      key
    ),
    cases = as.integer(cases)
  ) %>%
  separate(key, into = c("new", "type", "sexage"), sep = "_") %>%
  separate(sexage, into = c("sex", "age"), sep = 1) %>%
  select(-new, -iso2, -iso3) %T>%
# Finalmente, guardamos los datos ordenados en formato RDS y CSV
saveRDS(here("data", "who_tidy.rds")) %>%
```

```
write_csv(here("data", "who_tidy.csv")) %>%
  glimpse()
```

Rows: 76,046

Columns: 6

```
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
```

Ahora queremos convertir las columnas “type”, “sex” y “age” en factores. Empezamos construyendo las siguientes tablas de correspondencia:

```
who_type_lookup_tbl <- tibble(
  code = c("ep", "rel", "sn", "sp"),
  label = c(
    "Tuberculosis extrapulmonar",
    "Caso de recaída",
    "Tuberculosis pulmonar baciloscopía negativa",
    "Tuberculosis pulmonar baciloscopía positiva"
  )
)

who_sex_lookup_tbl <- tibble(
  code = c("f", "m"),
  label = c("Mujer", "Hombre")
)

who_age_lookup_tbl <- tibble(
  code = c(
    "014",
    "1524",
    "2534",
    "3544",
    "4554",
    "5564",
    "65"
  ),
  label = c(
    "0-14",
    "15-24",
    "25-34",
    "35-44",
    "45-54",
```

```
"55-64",
"65+"
)
)
```

Ahora podemos añadir las columnas factor para las variables correspondientes en el conjunto de la OMS:

```
who_factor <- who_tidy %>%
  convert_codes_to_factor(
    code_col = type,
    lookup_tbl = who_type_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  convert_codes_to_factor(
    code_col = sex,
    lookup_tbl = who_sex_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  convert_codes_to_factor(
    code_col = age,
    lookup_tbl = who_age_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  glimpse()
```

Rows: 76,046

Columns: 9

```
$ country      <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan"~
$ year        <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997~
$ type        <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp"~
$ sex         <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f"~
$ age         <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014"~
$ cases       <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128~
$ type_factor <fct> Tuberculosis pulmonar baciloscopía positiva, Tuberculosis ~
$ sex_factor  <fct> Hombre, Hombre, Hombre, Hombre, Hombre, Hombre, Hombre, Mu~
$ age_factor  <fct> 0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65+, 0-14, 15-24,~
```

Finalmente, queremos usar esta información para analizar el número de casos de tuberculosis por tipo y sexo. Empezamos generando el tibble de frecuencias correspondiente.

```
who_type_sex_tbl <- who_factor %>%
  count(type_factor, sex_factor, wt = cases, name = "cases") %>%
  print(n = Inf)
```

```
# A tibble: 8 x 3
  type_factor          sex_factor    cases
  <fct>             <fct>      <int>
1 Tuberculosis extrapulmonar      Mujer    941880
2 Tuberculosis extrapulmonar      Hombre  1044299
3 Caso de recaída                Mujer    1201596
4 Caso de recaída                Hombre   2018976
5 Tuberculosis pulmonar baciloscopia negativa Mujer    2439139
6 Tuberculosis pulmonar baciloscopia negativa Hombre   3840388
7 Tuberculosis pulmonar baciloscopia positiva Mujer    11324409
8 Tuberculosis pulmonar baciloscopia positiva Hombre   20586831
```

1.2. La “Gramática de los Gráficos” y ggplot2

El concepto de la “Gramática de los Gráficos” fue introducido por Leland Wilkinson en su libro de 1999 *The Grammar of Graphics* [8]. Proporciona un marco para entender cómo crear visualizaciones de forma sistemática. Descompone el proceso de construir una gráfica en componentes como datos, estéticas, geometrías, estadísticas, coordenadas y temas. El paquete `ggplot2` [4] implementa esta gramática en R, permitiendo construir visualizaciones complejas por capas. Para más información sobre su historia y aplicaciones, consulte: [1, 2].

1.2.1. Capas en ggplot2

En `ggplot2`, las gráficas se construyen añadiendo múltiples *capas* que definen los componentes de la visualización. Cada capa corresponde a un aspecto específico de la gráfica y, en conjunto, forman la composición completa. Estas capas incluyen:

1. **Datos:** El conjunto a visualizar (usualmente data frame o tibble). Es la fuente de información de la gráfica.
2. **Estéticas:** Mapeos que relacionan variables con propiedades visuales, por ejemplo:
 - Posición en los ejes x e y.
 - Color, relleno.
 - Tamaño, forma.
 - Transparencia.

Estos mapeos definen *qué* datos se muestran y *cómo* se representan visualmente.

3. **Geometrías:** Objetos geométricos que muestran los datos; por ejemplo:
 - `geom_point()` para diagramas de dispersión.
 - `geom_line()` para gráficas de líneas.
 - `geom_col()` para barras.
 - `geom_histogram()` para histogramas.

La geometría controla *cómo* se dibujan los datos.

4. **Transformaciones estadísticas:** Cálculos opcionales aplicados a los datos antes de graficar, como:

- `stat_bin()` para agrupar en histogramas.
- `stat_smooth()` para ajustar curvas suaves.

Estos son pasos de preprocesamiento para la visualización.

5. **Escalas:** Definen cómo se traducen los valores a propiedades visuales; también controlan marcas de ejes, leyendas y guías.
6. **Coordenadas:** El sistema usado, como cartesiano (`coord_cartesian()`), polar (`coord_polar()`) o invertido (`coord_flip()`).
7. **Facetado:** Métodos para dividir los datos en subconjuntos y mostrar múltiples gráficas en una cuadrícula:

- `facet_wrap()`
- `facet_grid()`

8. **Etiquetas:** `labs()` agrega títulos, etiquetas de ejes y pies.

9. **Temas:** `theme()` controla la apariencia general (fuentes, fondo, rejillas).

Cada capa puede combinarse y personalizarse para construir gráficas complejas y elegantes. Esta *gramática por capas* invita a pensar la gráfica como una composición de componentes independientes en lugar de un objeto monolítico.

1.2.2. Graficación con ggplot2

Ilustraremos estos conceptos graficando el número de casos de tuberculosis por tipo y sexo, usando el tibble de frecuencias que creamos antes.

```
who_type_sex_plot <- who_type_sex_tbl %>%
  ggplot(aes(x = type_factor, y = cases, fill = sex_factor)) +
  geom_col(position = "dodge") +
  c_scale_fill("C rose", "C blue") +
  labs(
    title = "Casos de tuberculosis por tipo y sexo",
    x = "Tipo de tuberculosis",
    y = "Casos",
    fill = "Sexo"
  ) +
  theme_krul()
```

Ahora podemos guardar y graficar:

```
ggsave(
  filename = here("images", "who_plot.png"),
```

```
plot = who_type_sex_plot,
width = 12,
height = 6,
dpi = 300
)
who_type_sex_plot
```

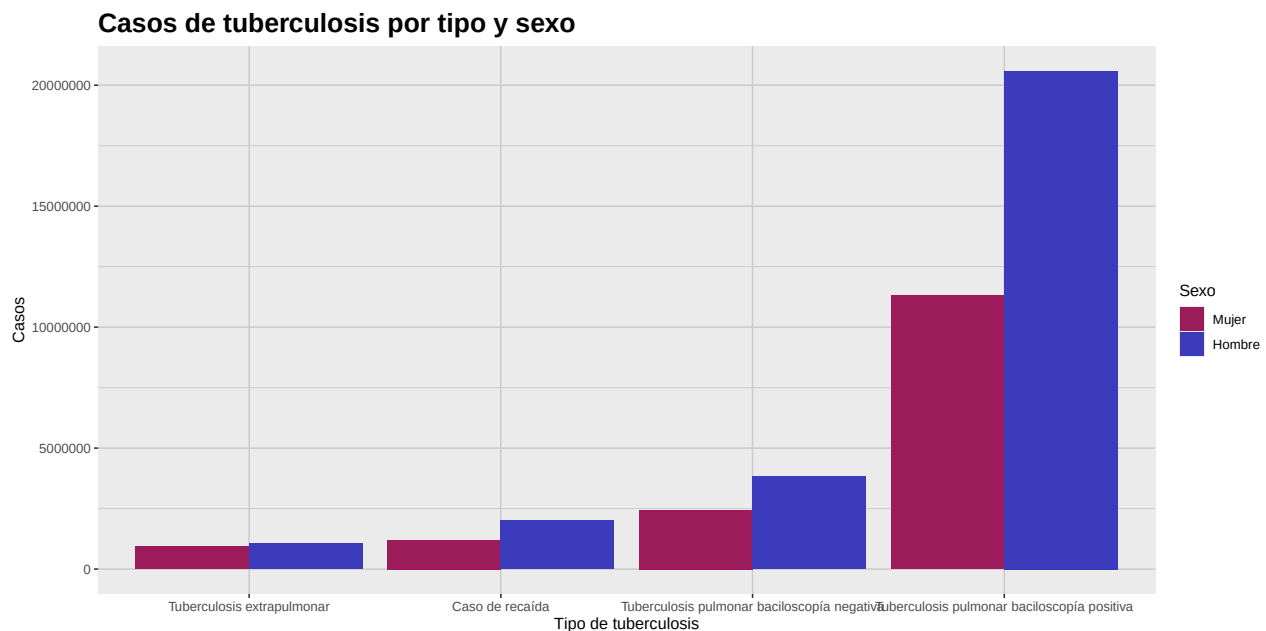


Figura 1.1: Esta gráfica muestra un análisis comparativo del número de casos de tuberculosis por tipo y sexo. Como se observa, el número de casos es consistentemente mayor en la población masculina en todos los tipos de tuberculosis.

1.3. Actividad: Limpieza de datos

1. En la sección anterior se mostró cómo usar el operador `%>%` para encadenar múltiples operaciones en el Tidyverse. En este ejercicio, muestre el resultado de todos los pasos intermedios en el proceso de ordenamiento del conjunto de la OMS, usando el operador pipe.
2. El problema con el conjunto *Billboard*, según los principios de "datos ordenados", es que las semanas en que una canción apareció en la lista están representadas como columnas, lo que dificulta analizar los datos. Utilizando las técnicas vistas hasta ahora, ordene el conjunto para analizar la evolución del ranking de la canción "Who Let The Dogs Out" de "Baha Men" en el Billboard Hot 100. La visualización final debe lucir como la que se muestra abajo.

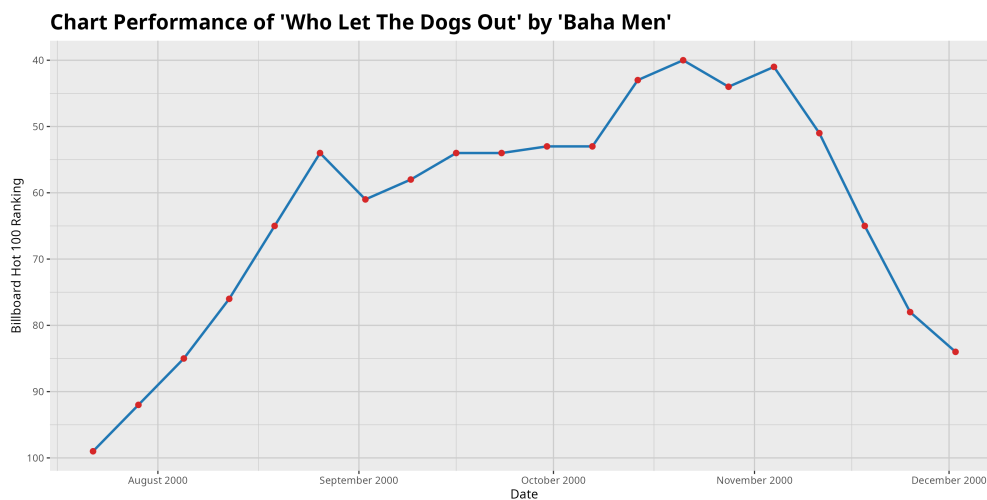


Figura 1.2: Evolución del ranking de la canción 'Who Let The Dogs Out' de 'Baha Men' en el Billboard Hot 100. La canción alcanzó el puesto 40 en la semana del 21 de octubre de 2000 y permaneció varias semanas en el top 100.

Bibliografía

- [1] Cleveland, W. S. (1993) *The Elements of Graphing Data*. Hobart Press.
- [2] Hyndman, R. J., and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- [3] Wickham, H. (2014) Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>
- [4] Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag.
- [5] Wickham, H., and Grolemund, G. (2017) *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O’Reilly Media.
- [6] Wickham, H. (2019) Functional programming with purrr. *UseR! Conference*.
- [7] Wickham, H., et al. (2019) Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- [8] Wilkinson, L. (1999) *The Grammar of Graphics*. Springer-Verlag.

Capítulo 2

Los momentos estandarizados

Como es bien sabido, cualquier distribución normal queda determinada por su media y desviación estándar. Sin embargo, muchos conjuntos de datos reales no siguen una distribución normal. En este capítulo exploraremos otras distribuciones y mostraremos cómo los conceptos de asimetría (skewness) y curtosis ayudan a entender su forma. También introduciremos los gráficos QQ, herramientas útiles para comparar la distribución de un conjunto de datos con una distribución teórica (usualmente la normal). Antes de comenzar, carguemos las librerías necesarias y fijemos algunas opciones.

```
library(tidyverse) # for data manipulation and visualization
library(broom) # for tidying data
library(here) # for file paths
library(patchwork) # for combining ggplots
library(krulRutils) # for custom package functions
library(magrittr) # for the pipe operator
library(e1071) # for skewness and kurtosis functions
library(car) # for robust scaling functions
library(ggforce) # for ggplot2 extensions
library(mvtnorm) # for multivariate normal distribution
library(MVN) # for multivariate normality tests
library(ellipse) # for generating covariance ellipses
library(ggrepel) # for better text annotations in ggplot2

options(scipen = 999)
```

2.1. Asimetría

Definición 2.1.1. La *asimetría* de una variable aleatoria X se define como el tercer momento estandarizado, dado por

$$\gamma_1 = \frac{\mu_3}{\sigma^3} = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3},$$

donde $\mu = \mathbb{E}[X]$ es la media y $\sigma = \sqrt{\mathbb{E}[(X - \mu)^2]}$ es la desviación estándar de X .

La asimetría mide la falta de simetría de una distribución. Una distribución es *positivamente asimétrica* (cola a la derecha) si posee una cola más larga a la derecha, y *negativamente asimétrica* (cola a la izquierda) si la cola más larga está a la izquierda. Una distribución perfectamente simétrica tiene asimetría cero. Para ilustrar estas ideas, introduciremos la distribución Gamma.

Definición 2.1.2. Decimos que una variable aleatoria continua X sigue una *distribución Gamma* con parámetro de forma $\alpha > 0$ y parámetro de tasa $\beta > 0$, si su función de densidad de probabilidad (PDF) está dada por

$$f_X(x; \alpha, \beta) = \begin{cases} \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, & x > 0, \\ 0, & x \leq 0, \end{cases} \quad (2.1)$$

donde $\Gamma(\alpha)$ denota la función Gamma, definida como

$$\Gamma(\alpha) = \int_0^\infty t^{\alpha-1} e^{-t} dt.$$

Entre otras aplicaciones, la distribución Gamma se usa frecuentemente para modelar el número total de reclamaciones de seguros en un periodo dado.

Comentario 2.1.3. La asimetría de una distribución Gamma está dada por

$$\gamma_1 = \frac{2}{\sqrt{\alpha}}.$$

Esto implica que la asimetría es siempre positiva y tiende a cero conforme α aumenta; es decir, la distribución se vuelve más simétrica al crecer α .

Mostraremos cómo graficar la PDF de la distribución Gamma para $\alpha = 2$ y $\beta = 1$, destacando la media y la mediana.

```
# Parameters
alpha <- 2
beta <- 1

# Create sequence of x values
x_data <- seq(0, 10, length.out = 200)

# Calculate density values
gamma_distribution_tbl <- tibble(
  x_data = x_data,
  density = dgamma(x_data, shape = alpha, rate = beta)
)

# Calculate mean and median
gamma_mean <- alpha / beta
gamma_median <- qgamma(0.5, shape = alpha, rate = beta)
```

```
gamma_sd <- sqrt(alpha) / beta

# Plot
gamma_distribution_plot <- gamma_distribution_tbl %>%
  ggplot(aes(x = x_data, y = density)) +
  geom_line(
    color = c_palette["C blue"],
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_mean,
    color = c_palette["C red"],
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_median,
    color = c_palette["C green"],
    linetype = "dotted",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = gamma_mean,
    y = 0.5,
    label = "Media",
    color = c_palette["C red"],
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = gamma_median,
    y = 0.5,
    label = "Mediana",
    color = c_palette["C green"],
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = paste(
      "PDF de la distribución Gamma (shape =", alpha, ", rate =", beta, ")"
    ),
    x = "X",
    y = "Densidad"
  ) +
```

```
theme_krul()
gamma_distribution_plot
```

PDF de la distribución Gamma (shape = 2 , rate = 1)

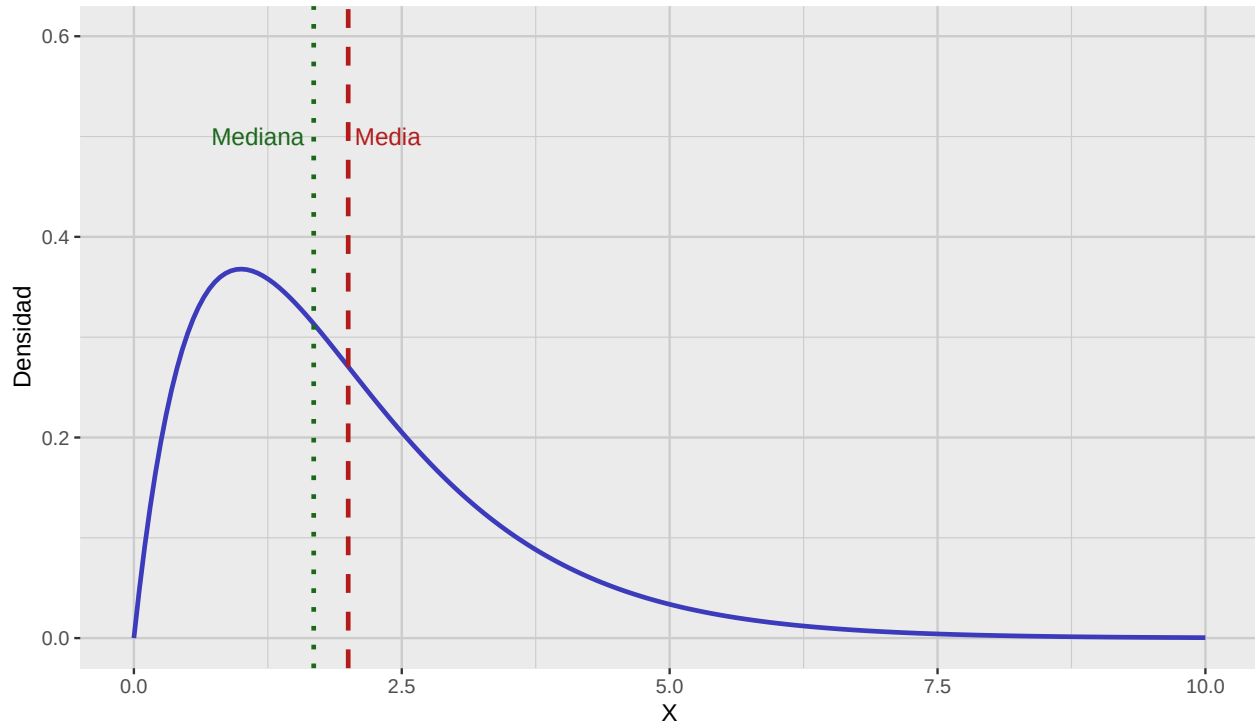


Figura 2.1: Función de densidad (PDF) de la distribución gamma con media y mediana destacadas.

2.2. Curtosis

Definición 2.2.1. La *curtosis* de una variable aleatoria X se define como el cuarto momento estandarizado, dado por

$$\beta_2 = \frac{\mu_4}{\sigma^4} = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4},$$

donde $\mu = \mathbb{E}[X]$ es la media y $\sigma = \sqrt{\mathbb{E}[(X - \mu)^2]}$ es la desviación estándar de X .

La curtosis mide el grosor de las colas de una distribución. Una distribución con alta curtosis (*leptocúrtica*) tiene colas más pesadas y un pico más agudo que la normal; con baja curtosis (*platicúrtica*) tiene colas más ligeras y un pico más plano. La curtosis de la normal es 3; por ello suele calcularse la *curtosis en exceso* dada por:

$$\gamma_2 = \beta_2 - 3.$$

Para ilustrar, introducimos la distribución de Laplace.

Definición 2.2.2. Decimos que una variable aleatoria continua X sigue una *distribución de Laplace* con parámetro de localización $\mu \in \mathbb{R}$ y de escala $b > 0$, si su PDF está dada por

$$f_X(x; \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x - \mu|}{b}\right), \quad x \in \mathbb{R}. \quad (2.2)$$

Entre otras aplicaciones, la distribución de Laplace se usa en procesamiento de imágenes y visión por computadora, particularmente para modelar ruido.

Comentario 2.2.3. La curtosis en exceso de una distribución de Laplace es

$$\gamma_2 = 3.$$

Esto significa que la distribución de Laplace tiene colas más pesadas que la normal.

Mostramos ahora cómo graficar la PDF de la distribución de Laplace con media cero y varianza uno.

```
# Parameters
mu <- 0 # mean (location)
b <- 1 / sqrt(2) # scale

# Sequence of x values covering enough range to see tails
x_data <- seq(-5, 5, length.out = 300)

# Laplace PDF function
dlaplace <- function(x, mu, b) {
  1 / (2 * b) * exp(-abs(x - mu) / b)
}

# Create data frame with PDF values
laplace_distribution_tbl <- tibble(
  x_data = x_data,
  density = dlaplace(x_data, mu, b)
)

# Plot PDF
laplace_distribution_tbl %>%
  ggplot(aes(x = x_data, y = density)) +
  geom_line(color = c_palette["C blue"], linewidth = 1) +
  labs(
    title = "PDF de la distribución de Laplace (media = 0, var = 1)",
    x = "x",
    y = "Densidad"
  ) +
  theme_krul()
```

PDF de la distribución de Laplace (media = 0, var = 1)

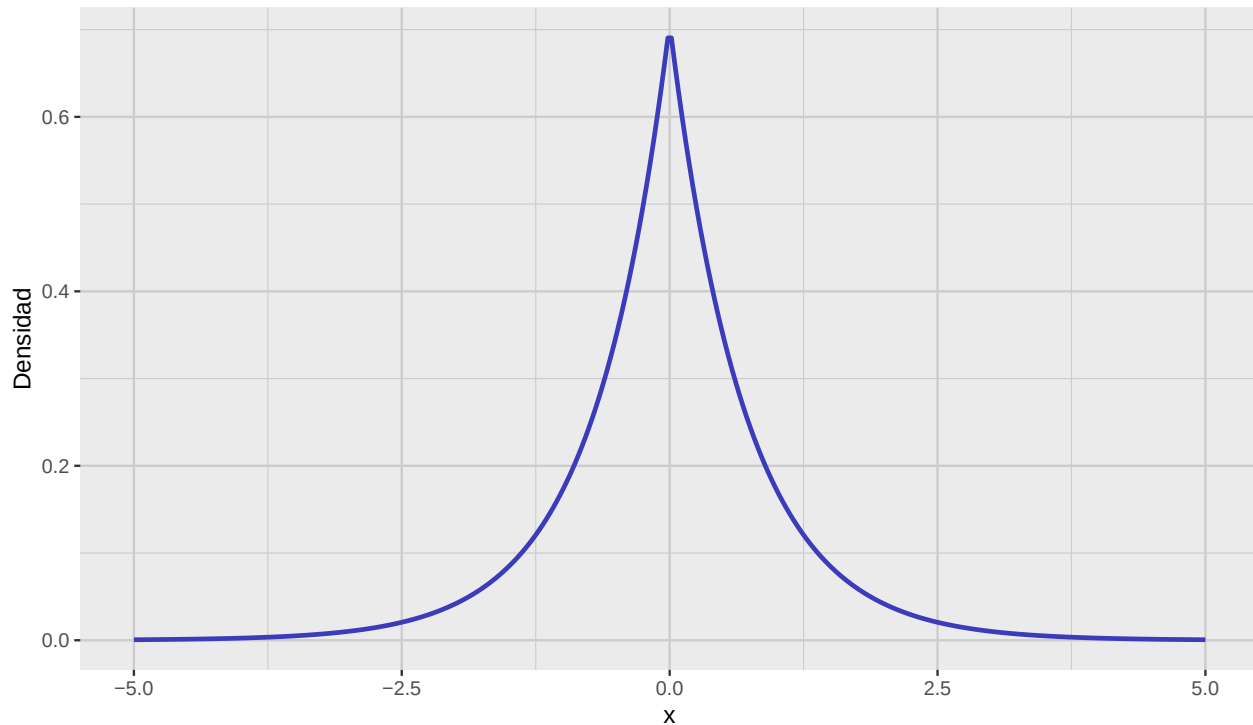


Figura 2.2: PDF de la distribución de Laplace con media cero y varianza uno.

2.3. Gráficos QQ

Los gráficos QQ (quantile–quantile) son herramientas para comparar la distribución de un conjunto de datos con una distribución teórica (por ejemplo, la normal). Son especialmente útiles para evaluar la normalidad.

Para construir un gráfico QQ, siga estos pasos:

1. **Ordene los datos muestrales:** Coloque los datos en orden creciente:

$$x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(n)}.$$

2. **Calcule posiciones de trazado (cuantiles empíricos):** Para cada valor ordenado $x_{(i)}$, calcule la probabilidad correspondiente

$$p_i = \frac{2i - 1}{2n}, \quad i = 1, 2, \dots, n.$$

3. **Obtenga los cuantiles teóricos de la normal:** A partir de la función cuantil (inversa de la CDF) de la normal estándar:

$$q_i = \Phi^{-1}(p_i),$$

donde Φ^{-1} es la función cuantil de la normal estándar.

4. **Trace los puntos:** Grafique los pares $(q_i, x_{(i)})$ con q_i en el eje x y $x_{(i)}$ en el eje y.
5. **Agregue una línea de referencia:** Trazar, por ejemplo, una recta por el primer y tercer cuartil ayuda a evaluar la normalidad.

Mientras más cerca caigan los puntos de esa recta, más se parece la muestra a la distribución normal especificada.

Para ilustrar estas ideas, generaremos muestras de las distribuciones Gamma y Laplace y construiremos sus gráficos QQ e histogramas. Primero, generamos las muestras:

```
set.seed(123) # for reproducibility

# Sample size
n <- 300
# Generate random samples from the Gamma distribution
gamma_sample <- rgamma(n, shape = alpha, rate = beta)

gamma_sample_tbl <- tibble(
  sample = gamma_sample
)

# Generate uniform random numbers in (0,1)
u <- runif(n)

# Apply inverse CDF of Laplace
laplace_sample <- ifelse(u < 0.5,
  mu + b * log(2 * u),
  mu - b * log(2 * (1 - u))
)

laplace_sample_tbl <- tibble(
  sample = laplace_sample
)
```

Ahora construiremos los correspondientes gráficos QQ e histogramas. Empezamos con las muestras de la distribución Gamma:

```
gamma_qq_plot <- gamma_sample_tbl %>%
  ggplot(aes(sample = sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
```

```
labs(  
  title = "Gráfico QQ",  
  x = "Cuantiles teóricos",  
  y = "Cuantiles muestrales"  
) +  
theme_krul()  
  
gamma_histogram <- gamma_sample_tbl %>%  
  ggplot(aes(x = sample)) +  
  geom_histogram(  
    bins = 30,  
    fill = c_palette["C blue"],  
    color = "black",  
    alpha = 0.7  
  ) +  
  labs(  
    title = "Histograma",  
    x = "Valores muestrales",  
    y = "Densidad"  
  ) +  
  theme_krul()  
  
gamma_plot <- gamma_qq_plot + gamma_histogram +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Distribución Gamma: gráfico QQ e histograma",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
  
gamma_plot
```

Distribución Gamma: gráfico QQ e histograma

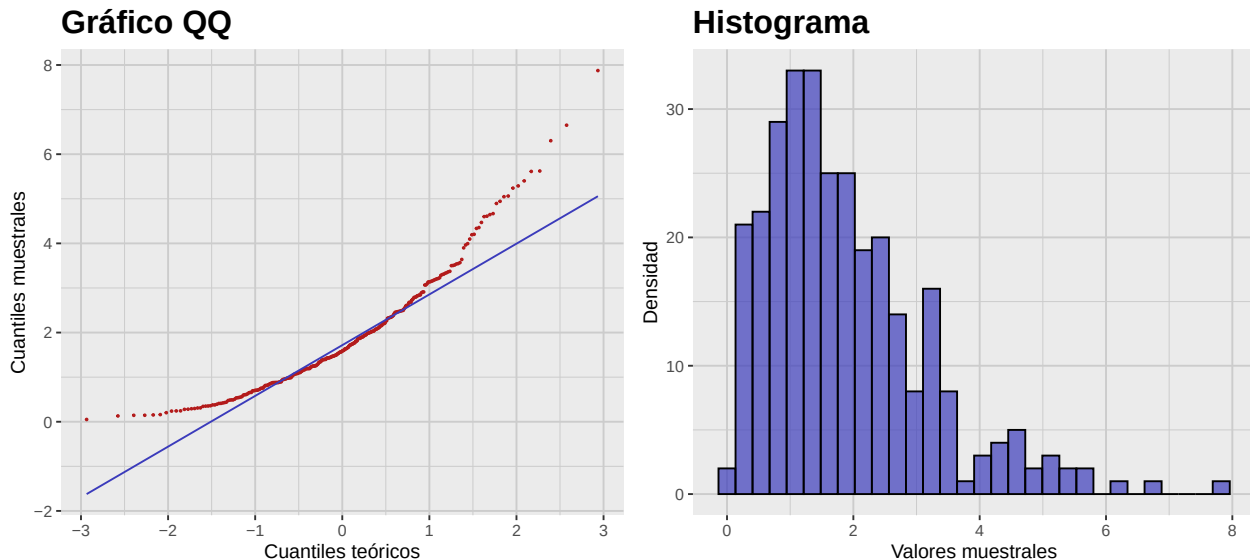


Figura 2.3: Gráfico QQ e histograma de las muestras de la distribución Gamma.

Construimos ahora el gráfico QQ y el histograma para las muestras de Laplace:

```
laplace_qq_plot <- laplace_sample_tbl %>%
  ggplot(aes(sample = sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "Gráfico QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

laplace_histogram <- laplace_sample_tbl %>%
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  )
```

```
) +
labs(
  title = "Histograma",
  x = "Valores muestrales",
  y = "Densidad"
) +
theme_krul()
laplace_plot <- laplace_qq_plot + laplace_histogram +
plot_layout(ncol = 2) +
plot_annotation(
  title = "Distribución de Laplace: gráfico QQ e histograma",
  theme = theme(
    plot.title = element_text(
      size = 24,
      face = "bold"
    )
  )
)
laplace_plot
```

Distribución de Laplace: gráfico QQ e histograma

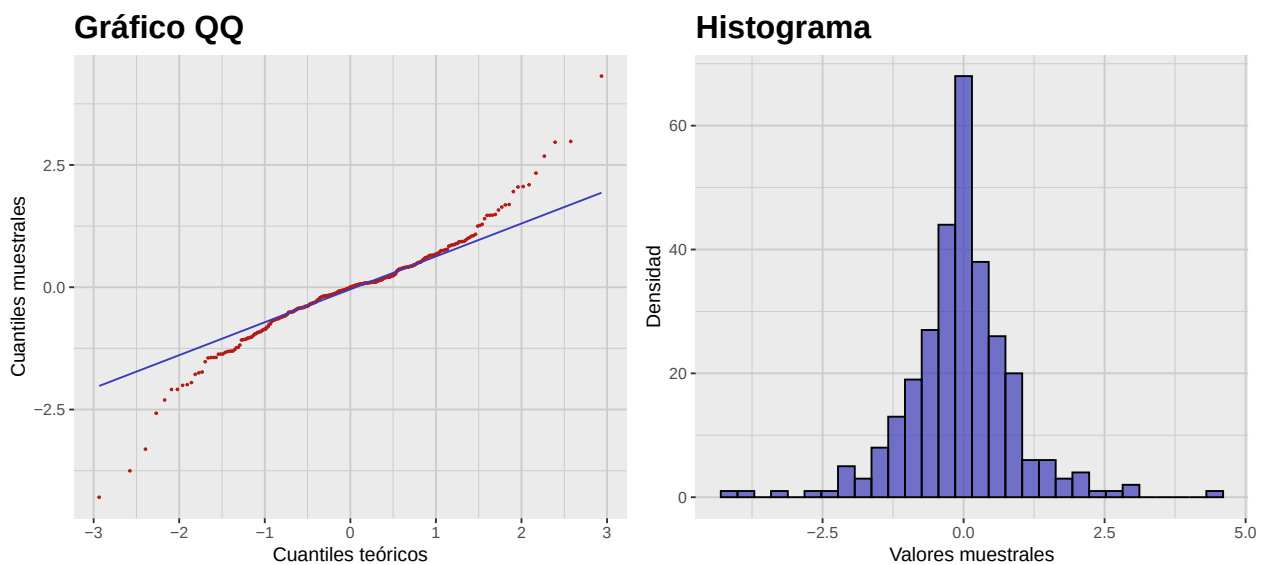


Figura 2.4: Gráfico QQ e histograma de las muestras de la distribución de Laplace.

2.4. Asimetría y curtosis muestrales

Hasta ahora hemos definido asimetría y curtosis para variables aleatorias. Sin embargo, en la práctica trabajamos con muestras y no con poblaciones completas. Por ello, necesitamos definir la asimetría y la curtosis *muestrales*.

Definición 2.4.1. Sean $x_1, x_2, \dots, x_n$ una muestra de tamaño n . La *media muestral* $\bar{x}$ y la *varianza muestral* s^2 están dadas por

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

La *asimetría muestral* g_1 está dada por

$$g_1 = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^3.$$

La *curtosis en exceso muestral* g_2 está dada por

$$g_2 = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{s} \right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}.$$

Como ejemplo, calcularemos la asimetría y la curtosis en exceso muestrales de las muestras de Gamma y Laplace generadas antes. Para la muestra Gamma tenemos:

```
# Compute sample skewness and excess kurtosis for Gamma distribution
gamma_sample_tbl %>%
  summarise(
    sample_skewness = skewness(sample, type = 2),
    sample_excess_kurtosis = kurtosis(sample, type = 2)
  ) %>%
  glimpse()
```

```
Rows: 1
Columns: 2
$ sample_skewness      <dbl> 1.293772
$ sample_excess_kurtosis <dbl> 2.164353
```

Para la muestra de Laplace tenemos:

```
# Compute sample skewness and excess kurtosis for Laplace distribution
laplace_sample_tbl %>%
  summarise(
    sample_skewness = skewness(sample, type = 2),
    sample_excess_kurtosis = kurtosis(sample, type = 2)
  ) %>%
  glimpse()
```

```
Rows: 1
Columns: 2
$ sample_skewness      <dbl> -0.07876743
$ sample_excess_kurtosis <dbl> 3.726374
```

2.5. Escalamiento y estandarización

En muchas situaciones, las unidades con que medimos distintas variables no son comparables (p.ej., estatura en cm y peso en kg). En esos casos conviene *escalar* las variables para hacerlas comparables. En esta sección discutimos los métodos más comunes para escalar y estandarizar variables y muestras.

2.5.1. Escalamiento de variables aleatorias

Definición 2.5.1. Una *transformación afín* de una variable aleatoria X es de la forma

$$Y = aX + b$$

donde a y b son constantes.

Comentario 2.5.2. En estadística, a menudo se habla de *escalamiento* (aunque incluya traslación) para referirse a transformaciones afines.

Suponga que X tiene media μ y desviación estándar σ . Entonces la variable

$$Y = aX + b$$

tendrá media $a\mu + b$ y desviación estándar $|a|\sigma$. En particular, si elegimos $a = 1/\sigma$ y $b = -\mu/\sigma$, la variable

$$Z = \frac{X - \mu}{\sigma}$$

tendrá media 0 y desviación estándar 1. Esta transformación se llama *estandarización*, y Z es la versión *estandarizada* de X (también llamada *puntaje z*).

Comentario 2.5.3. Si X no es normal, Z no será normal estándar. La estandarización no transforma una distribución no normal en normal.

Observación 2.5.4. Por definición,

$$\gamma_1(Z) = \mathbb{E}[Z^3] = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3} = \gamma_1(X)$$

y

$$\gamma_2(Z) = \mathbb{E}[Z^4] - 3 = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4} - 3 = \gamma_2(X).$$

Como ejemplo, consideremos de nuevo la distribución Gamma. En este caso

$$\mu = \frac{\alpha}{\beta} \quad \text{and} \quad \sigma = \frac{\sqrt{\alpha}}{\beta}.$$

Con esta información, generamos la gráfica de la versión estandarizada de la Gamma y la comparamos con la original.

```
# Calculate density values
z_gamma_distribution_tbl <- gamma_distribution_tbl %>%
  mutate(
    z_data = (x_data - gamma_mean) / gamma_sd,
    z_density = dgamma(x_data, shape = alpha, rate = beta) * gamma_sd
  ) %>%
  select(z_data, z_density)

# Calculate mean and median
z_gamma_mean <- 0
z_gamma_median <- (gamma_median - gamma_mean) / gamma_sd

# Plot
z_gamma_distribution_plot <- z_gamma_distribution_tbl %>%
  ggplot(aes(x = z_data, y = z_density)) +
  geom_line(
    color = c_palette["C blue"],
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_mean,
    color = c_palette["C red"],
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_median,
    color = c_palette["C green"],
    linetype = "dotted",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = z_gamma_mean,
    y = 0.55,
    label = "Media",
    color = c_palette["C red"],
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = z_gamma_median,
    y = 0.55,
    label = "Mediana",
    color = c_palette["C green"],
```

```
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = paste(
      "Estandarizada"
    ),
    x = "X",
    y = "Densidad"
  ) +
  theme_krul()

gamma_distribution_plot <- gamma_distribution_plot +
  labs(
    title = "Original"
  )

combined_gamma_plot <- gamma_distribution_plot + z_gamma_distribution_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Distribución Gamma: original y estandarizada",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

combined_gamma_plot
```


Distribución Gamma: original y estandarizada

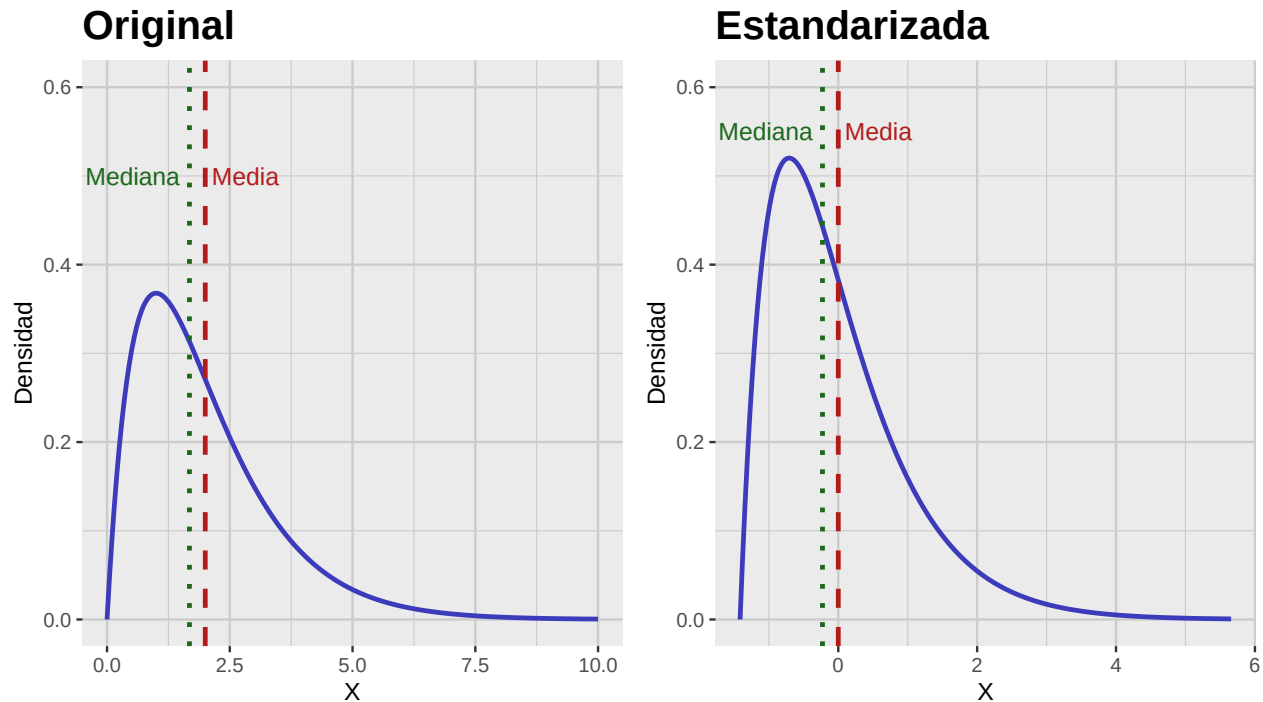


Figura 2.5: Comparación de la función de densidad (PDF) de la distribución Gamma original y su versión estandarizada.

2.5.2. Escalamiento de muestras

Para una muestra dada, no siempre se conoce la distribución subyacente; en particular, pueden desconocerse la media y la desviación poblacionales. Aun así, podemos escalar los datos; hay varios métodos comunes:

1. *Estandarización*: Usa la media y desviación muestrales. La muestra estandarizada es

$$T = \frac{X - \bar{X}}{S},$$

donde $\bar{X}$ es la media muestral y S la desviación muestral.

2. *Escalamiento Min-Max*: Escala a un rango fijo, usualmente $[0,1]$. La muestra escalada es

$$T = \frac{X - \min(X)}{\max(X) - \min(X)},$$

donde $\min(X)$ y $\max(X)$ son el mínimo y máximo de la muestra. Es muy sensible a atípicos al usar extremos.

3. *Escalamiento robusto*: Usa la mediana y el rango intercuartil (IQR). La muestra escalada es

$$T = \frac{X - \text{median}(X)}{\text{IQR}(X)},$$

donde $IQR(X) = Q_3 - Q_1$. Es más robusto a atípicos que Min-Max.

4. *Normalización*: Divide cada dato por la norma del vector. La muestra normalizada es

$$T = \frac{X}{\|X\|},$$

donde $\|X\|$ es la norma. Se usa con menor frecuencia que los anteriores.

Comentario 2.5.5. Como se mencionó, escalar no vuelve normal una distribución no normal. En particular, la asimetría y la curtosis en exceso muestrales no cambian.

Como ejemplo, consideraremos los distintos métodos de escalado aplicados a la muestra de la distribución Gamma que generamos anteriormente.

```
scale_lookup_tbl <- tibble(
  code = c(
    "sample",
    "sample_standard",
    "sample_min_max",
    "sample_robust",
    "sample_normal"
  ),
  label = c(
    "Original Sample",
    "Estandarizada",
    "Min-Max Scaled",
    "Robust Scaled",
    "Normalized"
  )
)

scaled_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    sample_standard = standard_scale(sample),
    sample_min_max = min_max_scale(sample),
    sample_robust = robust_scale(sample),
    sample_normal = normal_scale(sample)
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = "scaling_method",
    values_to = "sample_scaled"
  ) %>%
  convert_codes_to_factor(
    code_col = scaling_method,
    lookup_tbl = scale_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
```

```
    new_factor_col_name = scaling_method_factor
  )

scaled_gamma_qq_plot <- scaled_gamma_sample_tbl %>%
  ggplot(aes(sample = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "Gráficos QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

scaled_gamma_histogram <- scaled_gamma_sample_tbl %>%
  ggplot(aes(x = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogramas",
    x = "Valores muestrales",
    y = "Densidad"
  ) +
  theme_krul()
```

```
scaled_gamma_plot <- scaled_gamma_qq_plot + scaled_gamma_histogram +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Comparación de métodos de escalamiento aplicados a la muestra Gamma",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
scaled_gamma_plot
```

Comparación de métodos de escalamiento aplicados a la muestra Gamma

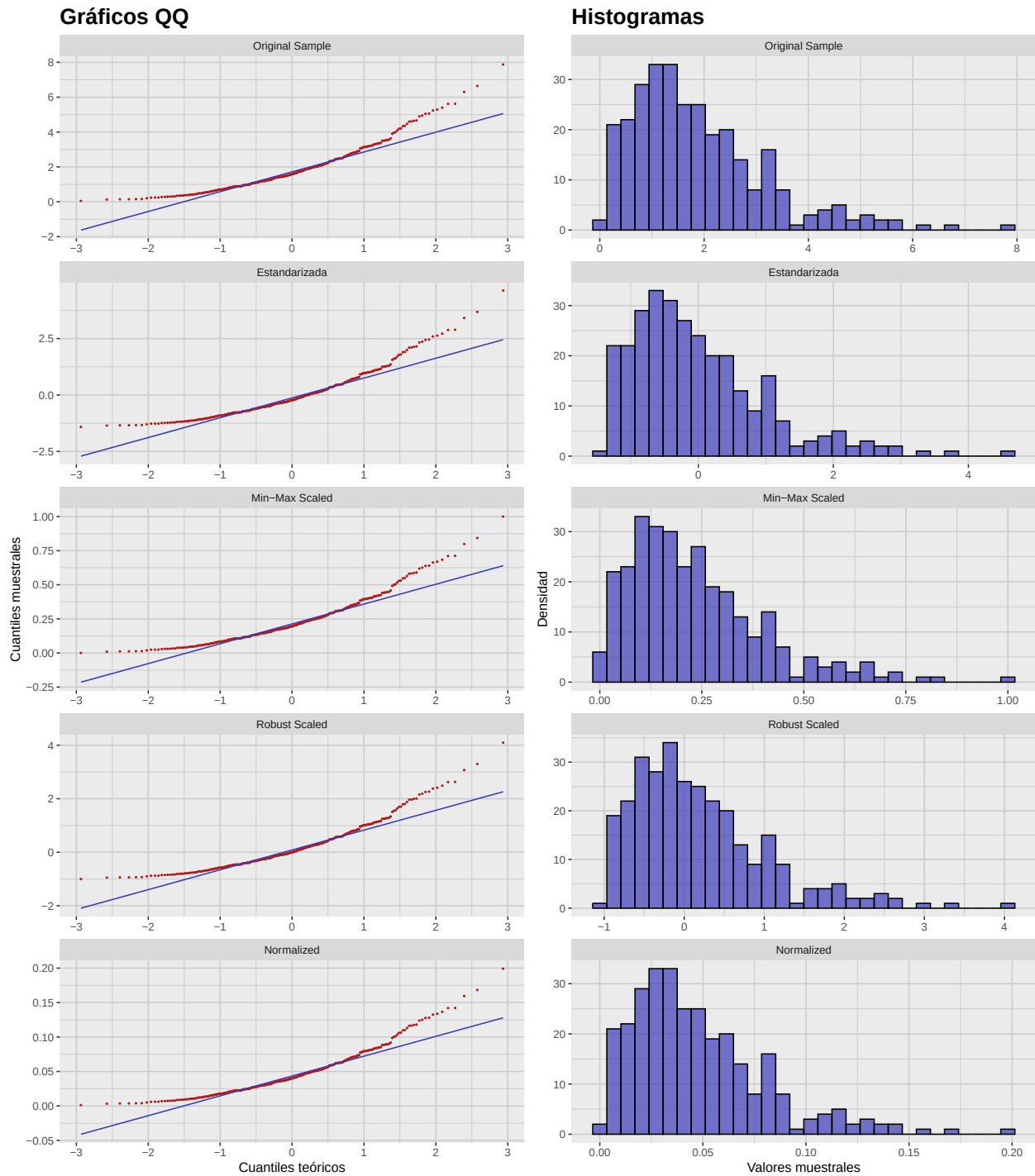


Figura 2.6: Comparación de distintos métodos de escalamiento aplicados a la muestra Gamma. Note que los QQ-plots son versiones reescaladas del gráfico QQ original y, por tanto, tienen la misma forma.

2.6. Actividad: Visualizando la no normalidad de muestras

Para cada una de las siguientes distribuciones, construya una muestra de tamaño 300 y calcule la asimetría y curtosis muestrales. Después, construya y compare sus gráficos QQ e histogramas.

1. Distribución exponencial con tasa $\lambda = 1$.
2. Distribución uniforme en $[0, 1]$.
3. Distribución beta con $\alpha = 2$ y $\beta = 5$.
4. Distribución de Cauchy con localización $x_0 = 0$ y escala $\gamma = 1$.
5. Distribución χ_k^2 con $k = 3$ grados de libertad.

Capítulo 3

Pruebas de normalidad

Aunque los gráficos QQ son una buena forma de evaluar visualmente la normalidad, pueden ser subjetivos e imprecisos. Por ello, es necesario contar con pruebas estadísticas más objetivas y precisas para determinar si los datos siguen una distribución normal. En este capítulo exploraremos pruebas de normalidad comunes y algunas transformaciones para acercar los datos a la normalidad.

3.1. Pruebas de normalidad

Existen varias pruebas estadísticas para evaluar si un conjunto sigue una normal. Algunas de las más usadas son:

1. *Shapiro-Wilk*: basada en la correlación entre los datos y puntajes normales. Es potente para muestras pequeñas (menores a 50) y muy usada; puede ser demasiado sensible en muestras grandes.
2. *Kolmogorov-Smirnov*: compara la función de distribución empírica con la acumulada de la normal. Es no paramétrica y aplicable a cualquier tamaño, pero menos potente que Shapiro-Wilk en muestras pequeñas; sensible a parámetros de escala y localización.
3. *Anderson-Darling*: similar a Kolmogorov-Smirnov, pero da mayor peso a las colas. Potente en muestras pequeñas y ampliamente utilizada.
4. *Jarque-Bera*: basada en la asimetría y curtosis de los datos. Potente en muestras grandes; en muestras pequeñas puede ser demasiado sensible a leves desvíos.

Aplicaremos estas pruebas a la muestra Gamma creada en el capítulo anterior:

```
normality_tests_lookup_tbl <- tibble(  
  code = c("shapiro", "ks", "ad", "jb"),  
  label = c(  
    "Shapiro-Wilk",  
    "Kolmogorov-Smirnov",  
    "Anderson-Darling",  
    "Jarque-Bera"  
  ),  
)
```

```
)

normality_test_gamma_sample_tbl <- gamma_sample_tbl %>%
  summarise(
    shapiro = tidy_shapiro_test(data = ., value_col = sample)$p_value,
    ks = tidy_ks_test(data = ., value_col = sample)$p_value,
    ad = tidy_ad_test(data = ., value_col = sample)$p_value,
    jb = tidy_jb_test(data = ., value_col = sample)$p_value
  ) %>%
  pivot_longer(
    everything(),
    names_to = "test",
    values_to = "p_value"
  ) %>%
  convert_codes_to_factor(
    code_col = test,
    lookup_tbl = normality_tests_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = test_factor
  ) %>%
  select(test_factor, p_value)
```

Los resultados de las pruebas de normalidad son los siguientes:

```
normality_test_gamma_sample_tbl
```

```
# A tibble: 4 x 2
  test_factor      p_value
  <fct>          <dbl>
1 Shapiro-Wilk    1.58e-12
2 Kolmogorov-Smirnov 7.93e- 3
3 Anderson-Darling 9.02e-16
4 Jarque-Bera      0
```

Como era de esperarse (la Gamma no es normal), ninguna prueba resulta satisfactoria.

3.2. Transformaciones

Para acercar los datos a la normalidad, pueden aplicarse transformaciones. Algunas comunes son:

1. *Transformación de Box-Cox*: familia de transformaciones potenciadas aplicables sólo a datos positivos. Está dada por:

$$\tilde{X} = \begin{cases} \frac{X^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \log(X) & \text{if } \lambda = 0 \end{cases}$$

donde λ es un parámetro.

2. *Transformación de Yeo-Johnson*: similar a Box-Cox, pero admite valores negativos; útil con asimetrías positivas o negativas. Está dada por:

$$\tilde{X} = \begin{cases} \frac{((X+1)^\lambda - 1)}{\lambda} & \text{if } X \geq 0, \lambda \neq 0 \\ \frac{-((-X+1)^{2-\lambda} - 1)}{2-\lambda} & \text{if } X < 0, \lambda \neq 2 \\ \log(X+1) & \text{if } X \geq 0, \lambda = 0 \\ \log(-X+1) & \text{if } X < 0, \lambda = 2 \end{cases}$$

donde λ es un parámetro.

Aplicaremos la transformación de Box-Cox a la muestra Gamma con $\lambda = 0, 0.25, 0.5, 0.75$ y 1:

```
box_cox_lookup_tbl <- tibble(
  code = c(
    "box_cox_0",
    "box_cox_025",
    "box_cox_05",
    "box_cox_075",
    "box_cox_1"
  ),
  label = c(
    "lambda = 0",
    "lambda = 0.25",
    "lambda = 0.5",
    "lambda = 0.75",
    "lambda = 1"
  ),
)

box_cox_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    box_cox_0 = bcPower(sample, lambda = 0),
    box_cox_025 = bcPower(sample, lambda = 0.25),
    box_cox_05 = bcPower(sample, lambda = 0.5),
    box_cox_075 = bcPower(sample, lambda = 0.75),
    box_cox_1 = bcPower(sample, lambda = 1)
  ) %>%
  pivot_longer(
    starts_with("box_cox_"),
    names_to = "lambda",
    values_to = "transformed_sample"
  ) %>%
  convert_codes_to_factor(
    code_col = lambda,
    lookup_tbl = box_cox_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  )
```

```
new_factor_col_name = lambda_factor
)

box_cox_gamma_qq_plot <- box_cox_gamma_sample_tbl %>%
  ggplot(aes(sample = transformed_sample)) +
  facet_wrap(
    ~lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "Gráficos QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

box_cox_gamma_histogram <- box_cox_gamma_sample_tbl %>%
  ggplot(aes(x = transformed_sample)) +
  facet_wrap(
    ~lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogramas",
    x = "Valores muestrales",
    y = "Densidad"
  ) +
  theme_krul()
```

```
box_cox_gamma_plot <- box_cox_gamma_qq_plot + box_cox_gamma_histogram +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Transformaciones Box-Cox con distintos valores de lambda",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
box_cox_gamma_plot
```

Transformaciones Box-Cox con distintos valores de lambda

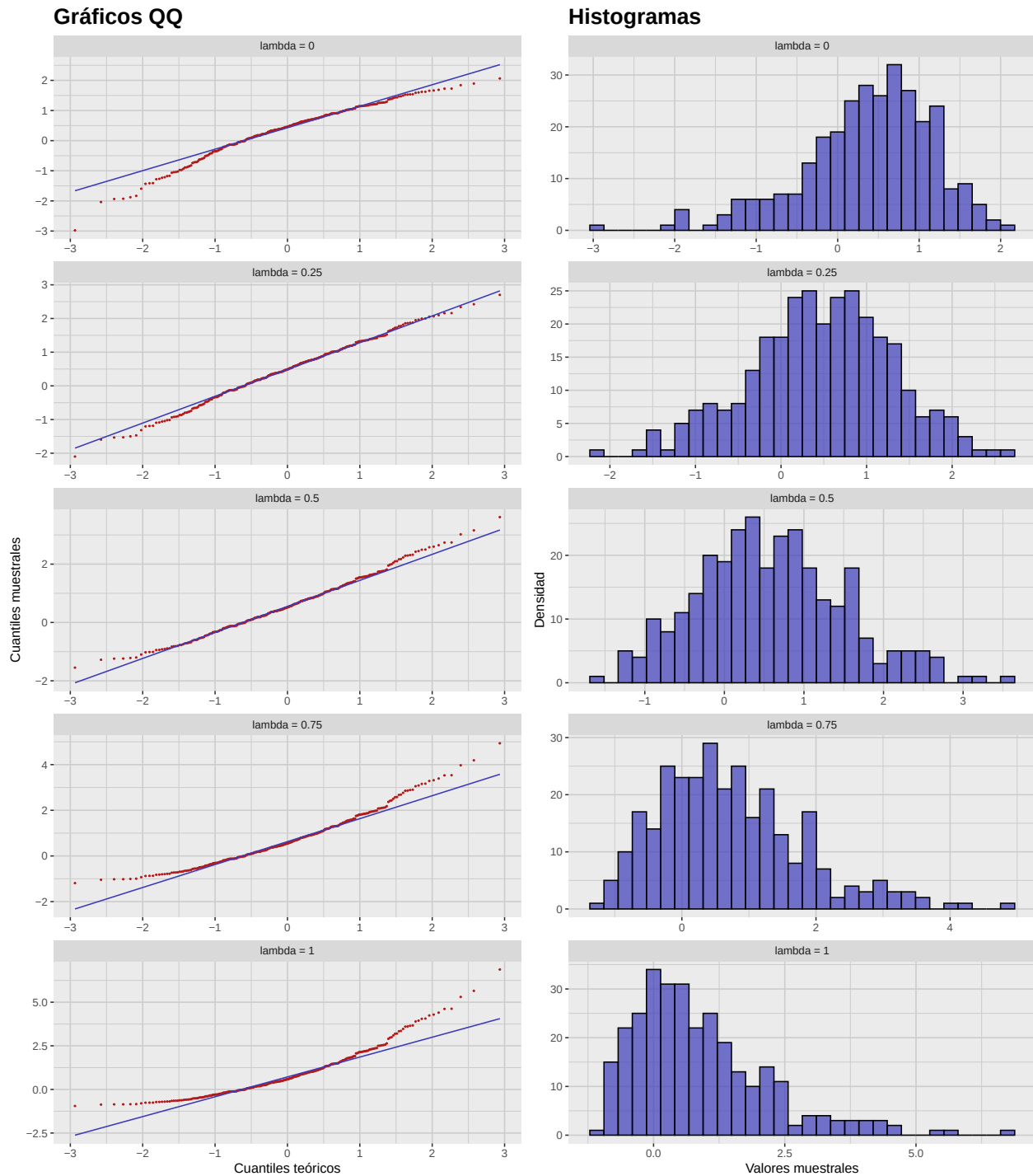


Figura 3.1: Comparación de distintas transformaciones de Box-Cox aplicadas a la muestra Gamma, con parámetro $\lambda = 0, 0.25, 0.5, 0.75$ y 1 .

3.3. Optimización de normalidad (Box–Cox)

Como vimos, la transformación de Box–Cox puede acercar los datos a la normalidad. La elección de λ es crucial: buscamos el valor que maximiza la log-verosimilitud. Esto puede realizarse con `car::powerTransform`, que estima el λ óptimo. La figura muestra el perfil de log-verosimilitud para la muestra Gamma:

```
# Intercept-only model
fit <- lm(sample ~ 1, data = gamma_sample_tbl)

# Estimate lambda
bc <- powerTransform(fit)

# Extract log-likelihood profile
bc_prof <- boxCox(fit, lambda = seq(-2, 2, 0.1), plotit = FALSE)

# Convert to tibble for ggplot
df <- tibble(lambda = bc_prof$x, loglik = bc_prof$y)

ggplot(df, aes(x = lambda, y = loglik)) +
  geom_line(color = c_pal("C blue"), linewidth = 1) +
  geom_vline(xintercept = bc$lambda, linetype = "dashed", color = c_pal("C red")) +
  labs(
    title = "Transformación de Box-Cox",
    x = expression(lambda),
    y = "Log-verosimilitud"
  ) +
  theme_krul()
```

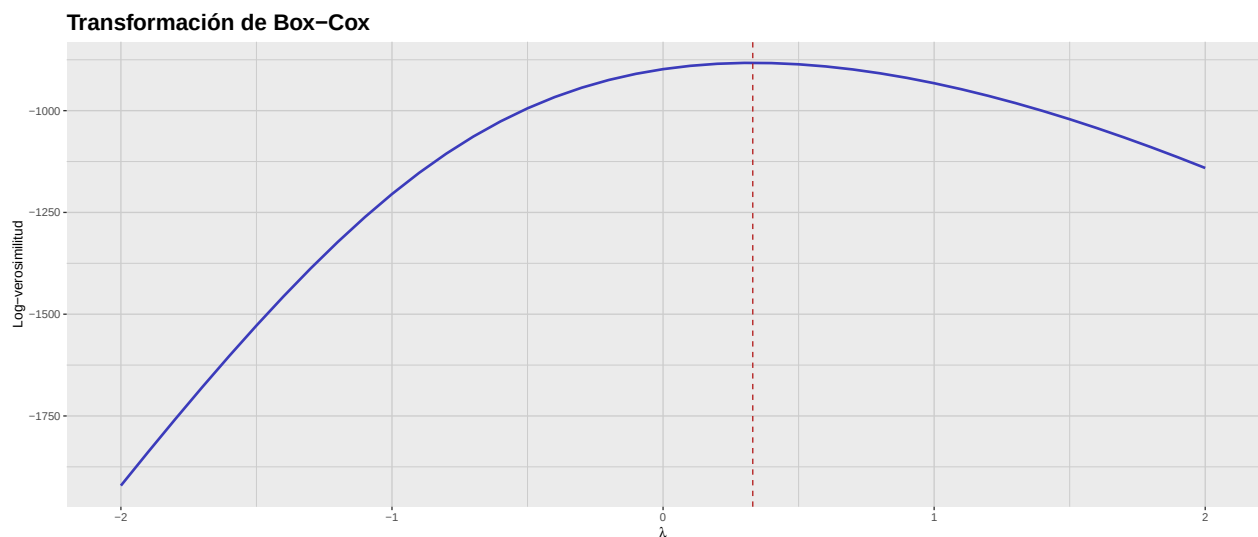


Figura 3.2: Perfil de log-verosimilitud para la transformación de Box–Cox aplicada a la muestra Gamma. La línea discontinua indica el valor óptimo de lambda.

Finalmente, extraemos el λ óptimo y transformamos los datos con ese valor:

```
optimal_lambda <- bc$lambda

optimal_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    transformed_sample = bcPower(sample, lambda = optimal_lambda)
  )

optimal_gamma_sample_tbl %>%
  summarise(
    shapiro = tidy_shapiro_test(data = ., value_col = transformed_sample)$p_value,
    ks = tidy_ks_test(data = ., value_col = transformed_sample)$p_value,
    ad = tidy_ad_test(data = ., value_col = transformed_sample)$p_value,
    jb = tidy_jb_test(data = ., value_col = transformed_sample)$p_value
  ) %>%
  pivot_longer(
    everything(),
    names_to = "test",
    values_to = "p_value"
  ) %>%
  convert_codes_to_factor(
    code_col = test,
    lookup_tbl = normality_tests_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = test_factor
  ) %>%
  select(test_factor, p_value)
```

```
# A tibble: 4 x 2
  test_factor      p_value
  <fct>           <dbl>
1 Shapiro-Wilk    0.974
2 Kolmogorov-Smirnov 1.000
3 Anderson-Darling 0.977
4 Jarque-Bera     0.878
```

Como se observa, la transformación de Box–Cox con el valor óptimo de λ mejora significativamente la normalidad de los datos, pues todas las pruebas arrojan valores p altos. A continuación se presentan el gráfico QQ y el histograma de los datos transformados:

```
optimal_gamma_qq_plot <- optimal_gamma_sample_tbl %>%
  ggplot(aes(sample = transformed_sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
```

```
) +  
geom_qq_line(  
  color = c_palette["C blue"],  
  linewidth = 0.5  
) +  
labs(  
  title = "Gráfico QQ",  
  x = "Cuantiles teóricos",  
  y = "Cuantiles muestrales"  
) +  
theme_krul()  
  
optimal_gamma_histogram <- optimal_gamma_sample_tbl %>%  
  ggplot(aes(x = transformed_sample)) +  
  geom_histogram(  
    bins = 30,  
    fill = c_palette["C blue"],  
    color = "black",  
    alpha = 0.7  
) +  
  labs(  
    title = "Histograma",  
    x = "Valores muestrales",  
    y = "Densidad"  
) +  
  theme_krul()  
  
optimal_gamma_plot <- optimal_gamma_qq_plot + optimal_gamma_histogram +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Transformación Gamma óptima",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
  
optimal_gamma_plot
```

Transformación Gamma óptima

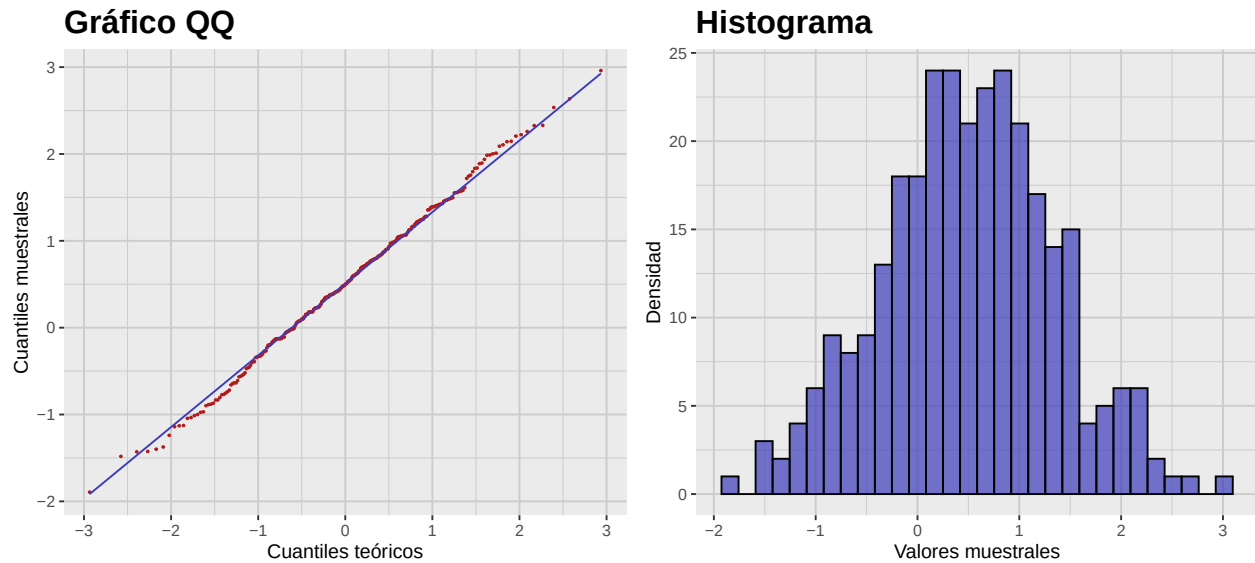


Figura 3.3: Gráfico QQ e histograma de la transformación Gamma óptima. Nótese el mejor ajuste a la normalidad en comparación con la muestra Gamma original.

Parte II

Análisis de Componentes Principales

Capítulo 4

Introducción al análisis multivariante

En este capítulo introducimos conceptos básicos del análisis multivariante, enfocándonos en el vector de medias y la matriz de covarianzas de un vector aleatorio. Estos conceptos son fundamentales para entender relaciones entre múltiples variables en un conjunto de datos.

4.1. El vector de medias y la matriz de covarianzas

Definición 4.1.1. Sea Π una población y sea

$$X : \Pi \rightarrow \mathbb{R}^p$$

un vector aleatorio (fila). El *vector de medias* de X se define como

$$\mu = \mathbb{E}[X] = [\mathbb{E}[X_1], \mathbb{E}[X_2], \dots, \mathbb{E}[X_p]] \quad (4.1)$$

donde X_i es la i -ésima componente del vector aleatorio X . La *matriz de covarianzas* de X se define como

$$\Sigma = \text{Cov}[X] = \mathbb{E}[(X - \mu)^T(X - \mu)] = [\text{Cov}[X_i, X_j]]_{i,j=1}^p \quad (4.2)$$

donde $\text{Cov}[X_i, X_j] = \mathbb{E}[(X_i - \mathbb{E}[X_i])(X_j - \mathbb{E}[X_j])]$ es la covarianza entre las componentes i -ésima y j -ésima del vector aleatorio X .

Comentario 4.1.2. Recuerde que $\text{Cov}[X_i, X_j] = \text{Cov}[X_j, X_i]$, por lo que la matriz es simétrica. Además, $\text{Cov}[X_i, X_i] = \text{Var}[X_i]$, la varianza de la i -ésima componente de X . Por ello también se le llama *matriz varianza-covarianza*.

Ahora, dada una muestra de tamaño n de Π , su *matriz muestral* es de tamaño $n \times p$ y cada renglón contiene una observación del vector X . En este contexto, el vector de medias muestral y la matriz de covarianzas muestral se definen así:

Definición 4.1.3. Dada una muestra de tamaño n de la población Π , con matriz muestral asociada x , definimos su *vector de medias muestral* como

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \left[\frac{1}{n} \sum_{i=1}^n x_{i,1}, \frac{1}{n} \sum_{i=1}^n x_{i,2}, \dots, \frac{1}{n} \sum_{i=1}^n x_{i,p} \right] \quad (4.3)$$

donde x_i es la fila i -ésima de la matriz muestral x y $x_{i,j}$ es el componente j -ésimo de la fila i -ésima. La *matriz de covarianzas muestral* se define como

$$\hat{\Sigma} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^T (x_i - \bar{x}) \quad (4.4)$$

donde $\hat{\Sigma}$ es una matriz $p \times p$ y $\hat{\Sigma}_{i,j} = \frac{1}{n-1} \sum_{k=1}^n (x_{k,i} - \bar{x}_i)(x_{k,j} - \bar{x}_j)$ es la covarianza muestral entre las componentes i -ésima y j -ésima del vector aleatorio X .

Comentario 4.1.4. La matriz de covarianzas muestral es un estimador insesgado de la matriz de covarianzas de X .

Ejemplo 4.1.5. Considere la siguiente matriz muestral de tamaño $n = 5$ de una población Π :

$$\begin{bmatrix} 2 & 5 & 7 \\ 4 & 3 & 6 \\ 6 & 4 & 5 \\ 8 & 2 & 4 \\ 10 & 1 & 3 \end{bmatrix}$$

Para obtener el vector de medias muestral, calcule:

$$\begin{aligned} \bar{x}_1 &= \frac{2 + 4 + 6 + 8 + 10}{5} = 6 \\ \bar{x}_2 &= \frac{5 + 3 + 4 + 2 + 1}{5} = 3 \\ \bar{x}_3 &= \frac{7 + 6 + 5 + 4 + 3}{5} = 5 \end{aligned}$$

En otras palabras,

$$\bar{x} = [6, 3, 5]$$

La varianza muestral de X_j es:

$$\hat{\Sigma}_{X_j} = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2$$

Por lo tanto,

$$\begin{aligned} \hat{\Sigma}_{X_1} &= \frac{(2-6)^2 + (4-6)^2 + (6-6)^2 + (8-6)^2 + (10-6)^2}{4} = 10 \\ \hat{\Sigma}_{X_2} &= \frac{(5-3)^2 + (3-3)^2 + (4-3)^2 + (2-3)^2 + (1-3)^2}{4} = 2.5 \\ \hat{\Sigma}_{X_3} &= \frac{(7-5)^2 + (6-5)^2 + (5-5)^2 + (4-5)^2 + (3-5)^2}{4} = 2.5 \end{aligned}$$

La covarianza muestral entre X_j y X_k es:

$$\hat{\Sigma}_{X_j, X_k} = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

Realizando los cálculos correspondientes, obtenemos:

$$\begin{aligned}
 \hat{\Sigma}_{X_1, X_2} &= \frac{(2-6)(5-3) + (4-6)(3-3) + (6-6)(4-3) + (8-6)(2-3) + (10-6)(1-3)}{4} \\
 &= \frac{(-8) + 0 + 0 + (-2) + (-8)}{4} = -4.5 \\
 \hat{\Sigma}_{X_1, X_3} &= \frac{(2-6)(7-5) + (4-6)(6-5) + (6-6)(5-5) + (8-6)(4-5) + (10-6)(3-5)}{4} \\
 &= \frac{(-8) + (-2) + 0 + (-2) + (-8)}{4} = -5 \\
 \hat{\Sigma}_{X_2, X_3} &= \frac{(5-3)(7-5) + (3-3)(6-5) + (4-3)(5-5) + (2-3)(4-5) + (1-3)(3-5)}{4} \\
 &= \frac{4 + 0 + 0 + 1 + 4}{4} = 2.25
 \end{aligned}$$

La matriz de covarianzas resultante es:

$$\hat{\Sigma} = \begin{bmatrix} 10 & -4.5 & -5 \\ -4.5 & 2.5 & 2.25 \\ -5 & 2.25 & 2.5 \end{bmatrix}$$

4.2. La distancia de Mahalanobis

Definición 4.2.1. Supongamos que tenemos una población Π con un vector aleatorio X que tiene vector de medias μ y matriz de covarianzas Σ . Si Σ es invertible, entonces la *distancia de Mahalanobis* entre dos puntos x e y en $\mathbb{R}^p$ se define como:

$$d_M(x, y) = \sqrt{(x - y)\Sigma^{-1}(x - y)^T} \quad (4.5)$$

En el contexto de datos muestrales, la distancia de Mahalanobis puede calcularse usando la matriz de covarianzas muestral $\hat{\Sigma}$:

Definición 4.2.2. Dada una muestra de tamaño n de la población Π , con matriz muestral asociada x y matriz de covarianzas muestral $\hat{\Sigma}$, la *distancia de Mahalanobis muestral* entre dos puntos x_i y x_j se define como:

$$d_M(x_i, x_j) = \sqrt{(x_i - x_j)\hat{\Sigma}^{-1}(x_i - x_j)^T} \quad (4.6)$$

Comentario 4.2.3. La distancia de Mahalanobis mide la separación entre dos puntos en un espacio multivariante, tomando en cuenta las correlaciones entre variables. Es particularmente útil para identificar atípicos en datos multivariantes.

Para ilustrar, consideremos el siguiente conjunto de datos. Iniciamos con una muestra aleatoria de tamaño $n = 300$ de una uniforme en $[0, 10]$.

```
set.seed(123)
n <- 300
x_data <- runif(n, min = 0, max = 10)
```

Construimos ahora un vector de error ϵ normal con media 0 y desviación 1:

```
epsilon_data <- rnorm(n, mean = 0, sd = 1)
```

Podemos entonces formar el vector

```
y_data <- x_data + epsilon_data
```

y crear el tibble:

```
sample_data_tbl <- tibble(  
  x = x_data,  
  y = y_data  
)
```

La dispersión asociada se muestra a continuación:

```
sample_data_plot <- sample_data_tbl %>%  
  ggplot(aes(x = x_data, y = y_data)) +  
  geom_point(  
    color = "black",  
    size = 0.2,  
    alpha = 1  
  ) +  
  labs(  
    title = "Datos de muestra",  
    x = "X",  
    y = "Y"  
  ) +  
  coord_fixed() +  
  theme_krul()  
  
sample_data_plot
```

Datos de muestra

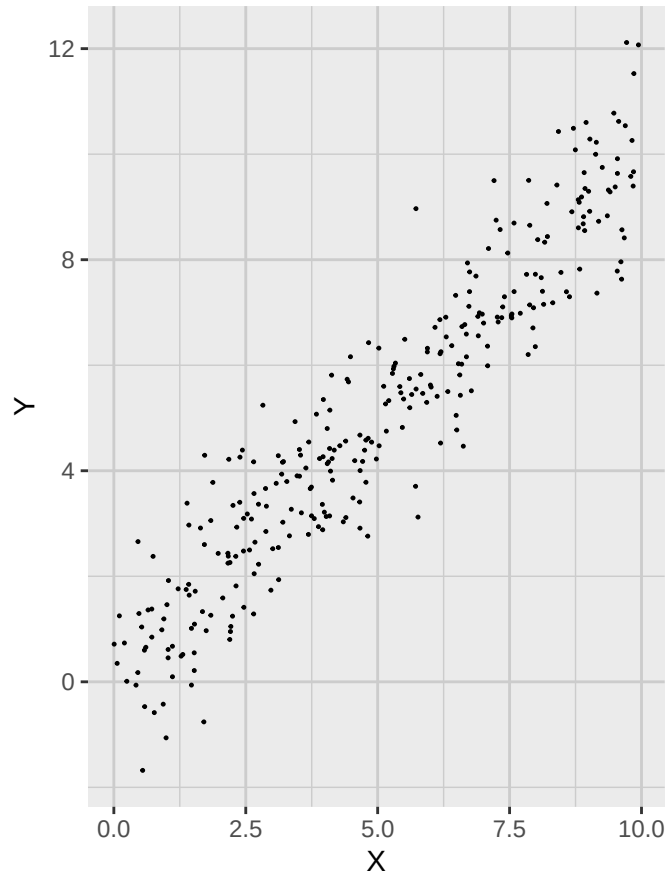


Figura 4.1: Diagrama de dispersión de los datos de muestra. Nótese la alta correlación entre las variables X e Y.

Ahora añadimos dos puntos extra a esta gráfica:

```
extra_points_tbl <- tibble(  
  x_extra = c(1, 5),  
  y_extra = c(1, 2)  
)  
  
extra_sample_data_plot <- sample_data_plot +  
  geom_point(  
    data = extra_points_tbl,  
    aes(x = x_extra, y = y_extra),  
    color = c_pal("C red"),  
    size = 1.5  
  )  
  
extra_sample_data_plot
```

Datos de muestra

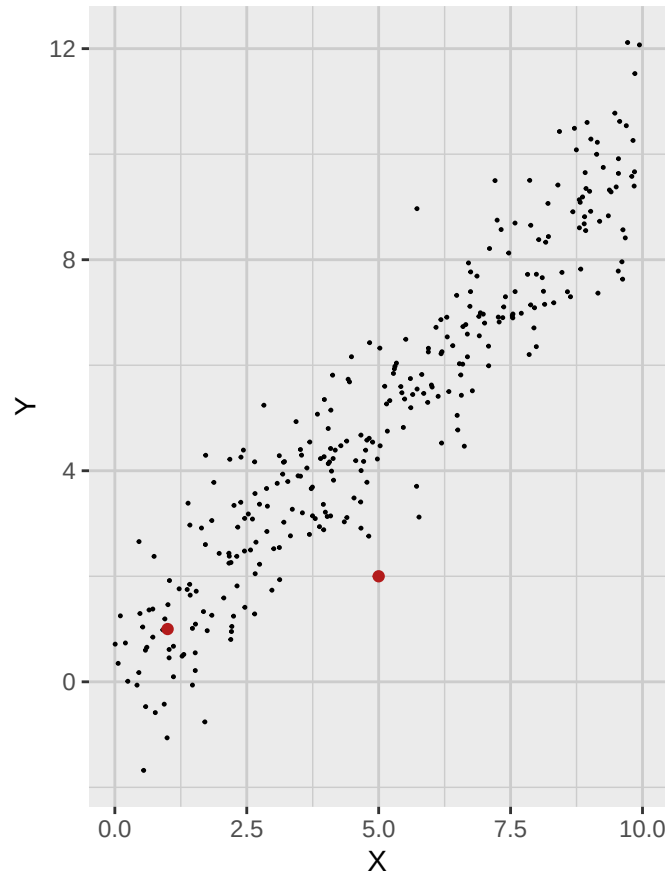


Figura 4.2: Dispersión con puntos adicionales. Nótese que uno de ellos es claramente atípico.

Nótese que uno de estos puntos es claramente un atípico. Nos gustaría tener una medida concreta para mostrarlo, así que calculamos el vector de medias muestral (centro de los datos). Para referencia futura, también calculamos la matriz de covarianzas muestral.

```
sample_mean_vector <- colMeans(sample_data_tbl)
sample_covariance_matrix <- cov(sample_data_tbl)

sample_mean_tbl <- tibble(
  x_bar = sample_mean_vector[1],
  y_bar = sample_mean_vector[2]
)
```

Si usamos la distancia euclídea, obtenemos que el punto atípico parece más cercano a la media muestral:

```
euclidean_sample_data_plot <- extra_sample_data_plot +
  geom_point(
    data = sample_mean_tbl,
```



```
    aes(x = x_bar, y = y_bar),  
    color = c_pal("C red"),  
    size = 1.5  
  ) +  
  geom_circle(  
    aes(x0 = sample_mean_vector[1], y0 = sample_mean_vector[2], r = 4),  
    color = c_pal("C blue"),  
    linewidth = 0.6,  
    fill = NA  
  )  
  
euclidean_sample_data_plot
```

Datos de muestra

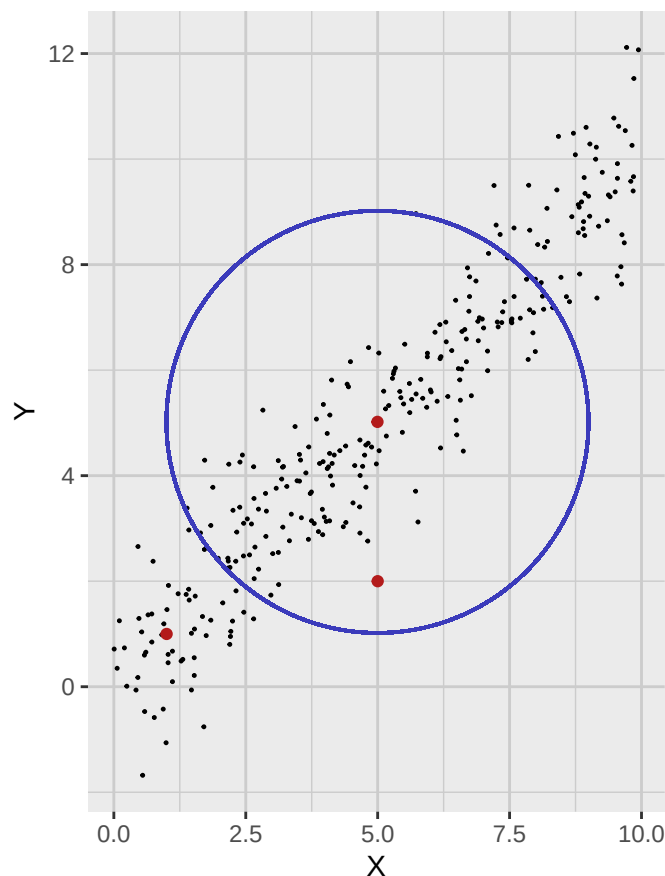


Figura 4.3: Dispersión con vector de media muestral y un círculo de distancia euclidiana constante al centro de los datos.

Por otra parte, si calculamos la distancia de Mahalanobis, obtenemos que el punto atípico está en realidad más alejado del vector de medias muestral:

```
annotated_sample_data_plot <- extra_sample_data_plot +
  geom_point(
    data = sample_mean_tbl,
    aes(x = x_bar, y = y_bar),
    color = c_pal("C red"),
    size = 1.5
  ) +
  stat_ellipse(
    type = "norm",
    level = 0.8647,
    color = c_pal("C blue"),
    linewidth = 0.6
  )
annotated_sample_data_plot
```

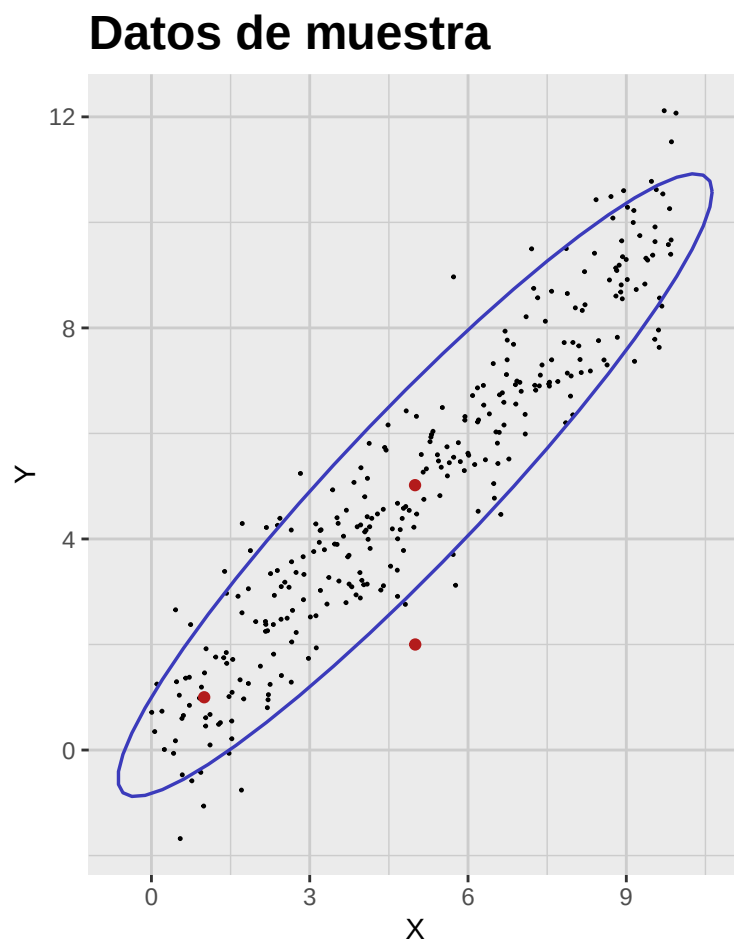


Figura 4.4: Dispersión con vector de media muestral y una elipse de distancia de Mahalanobis constante al centro de los datos.

4.3. Escalamiento y estandarización

Suponga ahora que tenemos un vector aleatorio X con media μ y covarianzas Σ . Definimos un nuevo vector Y como:

$$Y^T = AX^T + b$$

donde A es una matriz $p \times p$ y b un vector de dimensión p . El vector de medias y la matriz de covarianzas de Y se obtienen como:

$$\mu_Y^T = \mathbb{E}[Y]^T = A\mu^T + b$$

y

$$\Sigma_Y = \text{Cov}[Y] = A\Sigma A^T$$

Más adelante mostraremos que siempre es posible encontrar A y b tales que $\mu_Y = 0$ y $\Sigma_Y = I_p$ (identidad $p \times p$). A este proceso se le conoce como *estandarización* de X .

4.4. Actividad: La geometría de los datos multivariantes

Parte A: Ejercicios teóricos

1. Demuestre que la distancia de Mahalanobis se reduce a la euclídea si la covarianza es la identidad.
2. Muestre que la distancia de Mahalanobis es invariante bajo transformaciones afines.
3. Muestre que al estandarizar cada variable, la matriz de covarianzas pasa a ser la matriz de correlaciones.
4. Si X_1 y X_2 son normales estándar independientes, muestre que $d_M(X, 0)^2$ sigue una ji-cuadrada con 2 g.l., donde d_M es la distancia al origen.

Parte B: Ejercicios prácticos

1. Genere un conjunto 2D de 50 puntos (dos variables correlacionadas).
2. Calcule el vector de medias muestral y la matriz de covarianzas.
3. Escriba una función en R que calcule la distancia de Mahalanobis de cada punto a la media.
4. Identifique el punto con mayor distancia de Mahalanobis.
5. Grafique el conjunto con `ggplot2`.
6. Dibuje elipses de distancia de Mahalanobis 1, 2 y 3.
7. Coloree puntos según si su distancia supera 1, 2 o 3.
8. Cree un conjunto de 3 variables.
9. Calcule distancias de Mahalanobis en 3D.
10. Grafique pares de variables con elipses proyectadas.
11. Compare distancias euclídeas a la media con distancias de Mahalanobis.

Capítulo 5

Distribución normal multivariante

5.1. Definición y propiedades básicas

Definición 5.1.1. Sea μ un vector (renglón) y Σ una matriz simétrica definida positiva. Decimos que un vector aleatorio (renglón) X tiene *distribución normal multivariante* con media μ y matriz de covarianzas Σ , en símbolos $X \sim \mathcal{N}_p(\mu, \Sigma)$, si su densidad es

$$\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(X - \mu)\Sigma^{-1}(X - \mu)^T\right) dX$$

Comentario 5.1.2. Algunas observaciones sobre la definición:

1. Al igual que en el caso univariado, la normal multivariante queda determinada por μ y Σ .
2. Σ debe ser simétrica y definida positiva, garantizando que la distribución esté bien definida y el exponente sea negativo definido.
3. El término

$$|\Sigma| = \det(\Sigma)$$

es la *varianza generalizada*. Puede interpretarse como una medida de la “dispersión” en el espacio de dimensión p .

4. El término $(X - \mu)\Sigma^{-1}(X - \mu)^T$ es el cuadrado de la distancia de Mahalanobis entre X y la media μ .

Como en el caso univariado, la normal multivariante se destaca por el Teorema Central del Límite (TCL), que en términos generales afirma que la suma de muchas variables i.i.d. (debidamente estandarizadas) converge a una normal. El enunciado preciso es:

Teorema 5.1.3 (Teorema central del límite multivariante). *Sea $\{X_i\}_{i=1}^{\infty}$ una sucesión de vectores aleatorios independientes e idénticamente distribuidos en $\mathbb{R}^p$ con vector de medias μ y matriz de covarianzas Σ . Definimos la media muestral como*

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i.$$

Entonces, cuando $n \rightarrow \infty$, tenemos que

$$\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}_p(0, \Sigma),$$

donde $\xrightarrow{d}$ denota convergencia en distribución y $\mathcal{N}_p(0, \Sigma)$ es la normal multivariante p -dimensional con media 0 y covarianzas Σ .

Comentario 5.1.4. Aun cuando el TCL destaca a la normal multivariante, no es la única relevante en estadística multivariante. De hecho, muchos datos reales no siguen normalidad, y existen métodos para manejarla; además, técnicas como regresión lineal, PCA y clustering no arrojan buenos resultados aplicadas a una normal multivariante.

5.2. Contornos de la normal multivariante

Observe que la densidad de la normal multivariante es gaussiana en p dimensiones. Por ello, sus contornos son elipsoides centrados en μ y su forma está controlada por Σ . La probabilidad de estos contornos está gobernada por la χ^2 así: sea $0 \leq \varpi \leq 1$ y c tal que

$$\mathbb{P}(\chi_p^2 \leq c) = \varpi$$

Entonces, el contorno que contiene proporción ϖ de masa de probabilidad es el conjunto de X que satisfacen

$$(X - \mu)\Sigma^{-1}(X - \mu)^T = c$$

En la figura siguiente se muestran contornos de normales bivariantes con distintas covarianzas. Son elipses centradas en $\mu = (0, 0)$; el gradiente de color representa la densidad (más oscuro, mayor densidad).

```
# Add seed for reproducibility
set.seed(123)

# Parameters
mu <- c(0, 0)
Sigma_1 <- matrix(c(1, 0, 0, 1), nrow = 2)
Sigma_2 <- matrix(c(0.3, 0, 0, 1.5), nrow = 2)
Sigma_3 <- matrix(c(1, 0.6, 0.6, 1), nrow = 2)
Sigma_4 <- matrix(c(1, -0.6, -0.6, 1), nrow = 2)

# Function to generate contour + scatter plot
plot_bivariate_normal <- function(mu, Sigma, n_samples = 1000,
                                   xlim = c(-3, 3), ylim = c(-3, 3),
                                   grid_length = 100) {

  # Create grid
  x <- seq(xlim[1], xlim[2], length.out = grid_length)
  y <- seq(ylim[1], ylim[2], length.out = grid_length)
  grid <- expand.grid(x = x, y = y)
```

```
grid$z <- dmvnorm(grid, mean = mu, sigma = Sigma)

# Sample points
samples <- rmvnorm(n_samples, mean = mu, sigma = Sigma)
colnames(samples) <- c("x", "y")
samples <- as_tibble(samples)

# Plot
normal_plot <- ggplot() +
  geom_contour(data = grid, aes(x, y, z = z, color = after_stat(level))) +
  geom_point(
    data = samples,
    aes(x, y),
    color = c_pal("C red"),
    alpha = 0.5,
    size = 0.1
  ) +
  scale_color_viridis_c(option = "mako") +
  coord_cartesian(xlim = xlim, ylim = ylim) +
  labs(
    title = "",
    x = expression(X[1]), y = expression(X[2]),
    color = "Densidad"
  ) +
  theme_krul()

return(normal_plot)
}

normal_plot_1 <- plot_bivariate_normal(mu, Sigma_1)
normal_plot_2 <- plot_bivariate_normal(mu, Sigma_2)
normal_plot_3 <- plot_bivariate_normal(mu, Sigma_3)
normal_plot_4 <- plot_bivariate_normal(mu, Sigma_4)

normal_plot <- (normal_plot_1 + normal_plot_2) /
  (normal_plot_3 + normal_plot_4) +
  plot_layout(widths = c(1, 1), heights = c(1, 1)) +
  plot_annotation(
    title = "Contornos de la normal bivalente ",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

)
`normal_plot`

Contornos de la normal bivalente

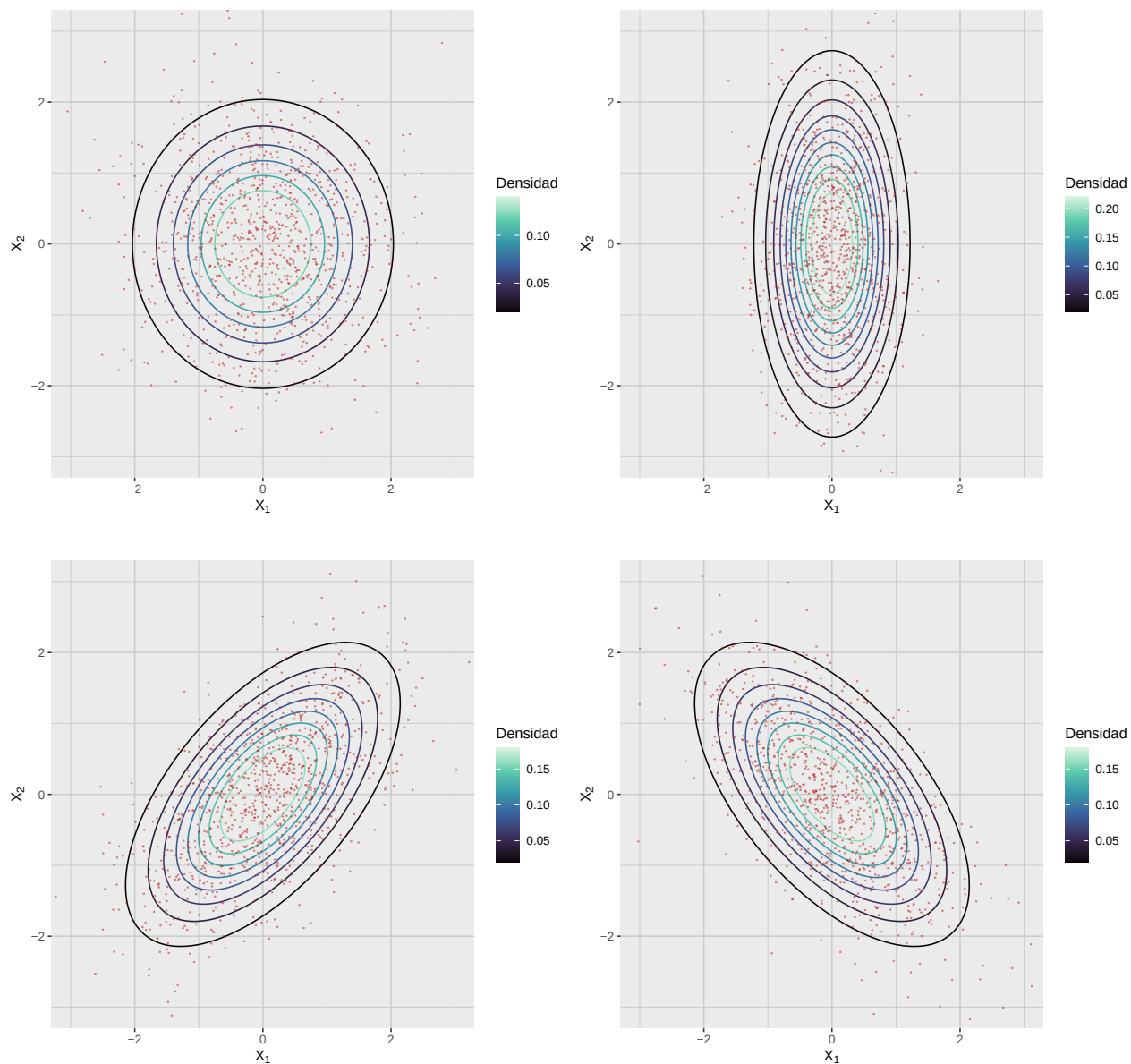


Figura 5.1: Distribución normal bivalente con distintas matrices de covarianzas

5.3. Estandarización de la normal multivariante

Al igual que en el caso univariado, si $X \sim \mathcal{N}_p(\mu, \Sigma)$ podemos estandarizar para obtener $Z \sim \mathcal{N}_p(0, I_p)$, donde I_p es la identidad de tamaño p . Para describirlo, recordemos algunos resultados de álgebra lineal.

Teorema 5.3.1 (Descomposición en valores singulares). *Sea Σ una matriz simétrica definida positiva de dimensión $p \times p$. Entonces, existe una matriz ortogonal Q y una matriz diagonal Λ con entradas positivas tal que*

$$\Sigma = Q\Lambda Q^T$$

Nota 5.3.2. Recordemos que una matriz Q es ortogonal si $Q^T Q = I_p$; sus columnas son ortonormales y forman una base de $\mathbb{R}^p$.

Comentario 5.3.3. La descomposición en valores singulares (SVD) permite descomponer una matriz en valores (diagonal de Λ , también *autovalores*) y vectores singulares (columnas de Q , *autovectores*). Es útil para entender la geometría de la normal multivariante.

Como Σ es definida positiva, las entradas diagonales de

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_p \end{bmatrix}$$

son todas positivas. En particular, podemos definir una nueva matriz

$$D = \begin{bmatrix} \sqrt{\lambda_1} & 0 & \cdots & 0 \\ 0 & \sqrt{\lambda_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sqrt{\lambda_p} \end{bmatrix}$$

que satisface la condición $D^2 = \Lambda$. Si ahora tomamos

$$\sigma = DQ^T$$

entonces

$$\Sigma = \sigma^T \sigma$$

En particular, podemos definir la *standarización* del vector aleatorio X como

$$Z = (X - \mu)\sigma^{-1} = (X - \mu)QD^{-1} = (X - \mu)Q\Lambda^{-1/2}$$

en cuyo caso $Z \sim \mathcal{N}_p(0, I_p)$.

5.4. Interpretación Geométrica de la SVD

Anteriormente en este capítulo consideramos la distribución normal multivariante con media

$$\mu = [0, 0]$$

y matriz de covarianzas

$$\Sigma = \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix}$$

De acuerdo con la SVD, podemos escribir esta matriz de covarianzas como

$$\Sigma = Q\Lambda Q^T$$

para alguna matriz ortogonal Q y diagonal Λ . En este caso, la SVD de Σ se calcula así:

```
# Example mean and covariance
mu <- c(0, 0)
Sigma <- matrix(c(
  1, 0.6,
  0.6, 1
), 2, 2)

# Compute eigenvectors and eigenvalues
eig <- eigen(Sigma)
vectors <- eig$vectors
values <- eig$values
```

Obtenemos:

$$Q = \begin{bmatrix} 0.7071068 & -0.7071068 \\ 0.7071068 & 0.7071068 \end{bmatrix}$$

y

$$\Lambda = \begin{bmatrix} 1.6 & 0 \\ 0 & 0.4 \end{bmatrix}$$

donde los vectores singulares son las columnas de Q y los valores singulares son las entradas diagonales de Λ . Geométricamente, los vectores singulares representan las direcciones de los ejes del elipsoide que define los contornos de la normal multivariante, y las raíces cuadradas de los valores singulares representan las longitudes de dichos ejes.

```
# Scale eigenvectors by sqrt(eigenvalues) for plotting
vec_data <- tibble(
  x0 = mu[1],
  y0 = mu[2],
  x1 = mu[1] + sqrt(values) * vectors[1, ],
  y1 = mu[2] + sqrt(values) * vectors[2, ]
)

# Generate points of the 1-Mahalanobis-distance ellipse
ellipse_points <- as_tibble(ellipse(Sigma, centre = mu, level = 0.393))
# level = 0.393 corresponds to ~1 standard deviation for 2D
```

```
# Plot
ggplot() +
  geom_path(
    data = ellipse_points,
    aes(x = x, y = y),
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_segment(
    data = vec_data,
    aes(x = x0, y = y0, xend = x1, yend = y1),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C green"),
    linewidth = 1
  ) +
  geom_point(
    aes(x = mu[1], y = mu[2]),
    color = c_pal("C red"),
    size = 3
  ) +
  coord_equal() +
  labs(
    title = "Interpretación geométrica de la SVD",
    x = expression(X[1]),
    y = expression(X[2])
  ) +
  theme_krul()
```

Interpretación geométrica de la SVD

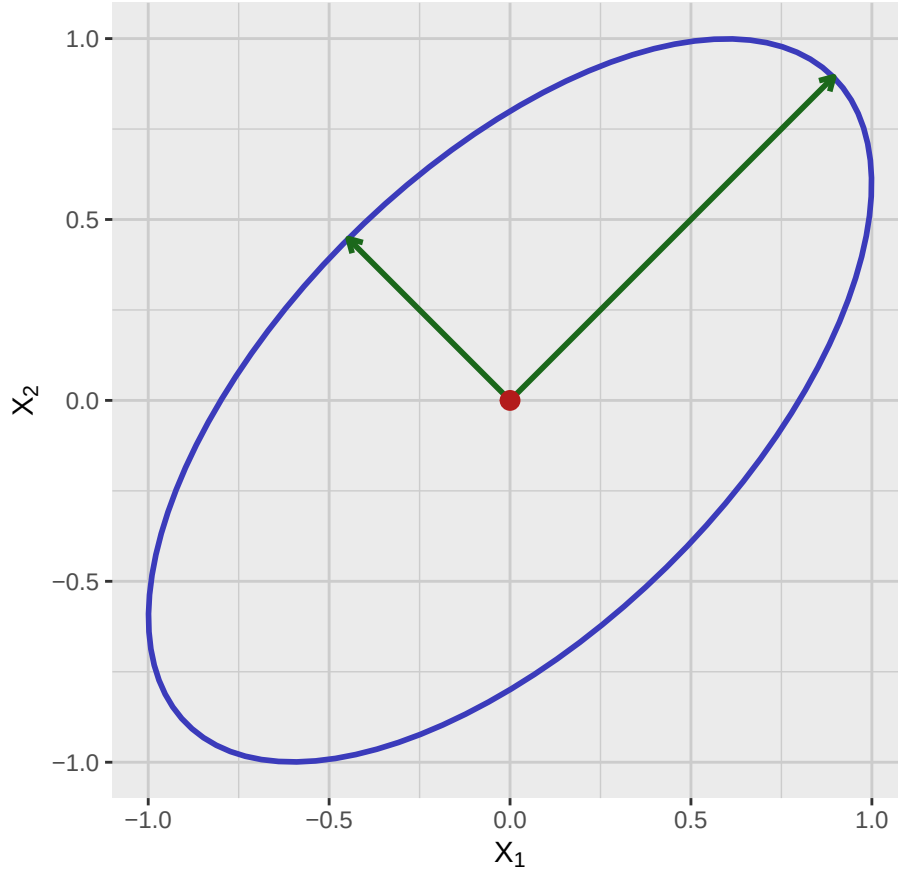


Figura 5.2: Interpretación geométrica de la SVD de una normal bivalente. El punto rojo es la media, las flechas verdes son los eigenvectores escalados por la raíz de los eigenvalores y la elipse azul es el contorno de distancia de Mahalanobis igual a 1.

5.5. Clausura bajo transformaciones lineales

Teorema 5.5.1 (Clausura bajo transformaciones lineales). Sea $X \sim \mathcal{N}_p(\mu, \Sigma)$ un vector con distribución normal multivariante. Para cualquier matriz A de tamaño $m \times p$ y vector b de tamaño m , el vector aleatorio

$$Y = AX + b$$

también tiene distribución normal multivariante: $Y \sim \mathcal{N}_m(A\mu + b, A\Sigma A^T)$.

Comentario 5.5.2. Este teorema afirma que si aplicamos una transformación lineal a un vector con distribución normal multivariante, el resultado también es normal multivariante. Las expresiones de media y covarianza se dan en el enunciado. Es especialmente útil en aplicaciones como regresión (manteniendo normalidad de residuos) y para derivar distribuciones de combinaciones lineales.

Ejemplo 5.5.3. Sea $X \sim \mathcal{N}_2(\mu, \Sigma)$ un vector aleatorio con media $\mu = (1, 2)$ y matriz de covarianzas

$$\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix}$$

Considere la transformación lineal

$$Y = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} X + \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Es decir,

$$\begin{aligned} Y_1 &= 2X_1 + X_2 \\ Y_2 &= 3X_2 + 1 \end{aligned}$$

Usando el teorema de clausura bajo transformaciones lineales, obtenemos la media y la covarianza de Y :

$$\begin{aligned} \text{Mean: } A\mu + b &= \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \end{bmatrix} \\ \text{Covariance: } A\Sigma A^T &= \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 8 & 9 \\ 9 & 18 \end{bmatrix} \end{aligned}$$

Así,

$$Y \sim \mathcal{N}_2 \left(\begin{bmatrix} 4 \\ 7 \end{bmatrix}, \begin{bmatrix} 8 & 9 \\ 9 & 18 \end{bmatrix} \right)$$

Comentario 5.5.4. Nótese que la proyección a un subespacio es lineal, por lo que el teorema de clausura implica que cualquier subconjunto de componentes de una normal multivariante también es normal multivariante. Esto permite analizar subconjuntos de variables manteniendo la suposición de normalidad.

5.6. Pruebas de normalidad multivariante

Existen varias pruebas para verificar si un conjunto sigue normalidad multivariante. Son esenciales, pues muchos métodos la asumen. Algunas de las más usadas y su implementación en R:

1. **Prueba de Mardia:** Basada en asimetría y curtosis multivariantes. Para normalidad multivariante, la asimetría debe ser cercana a 0 y la curtosis a $p(p+2)$, donde p es el número de variables. En R: `MVN::mvn(datos, mvn_test = "mardia")`.
2. **Prueba de Henze–Zirkler (HZ):** Usa la función característica empírica para medir desviaciones de la normalidad. Tiene buen poder global y es adecuada en dimensiones moderadas. En R: `MVN::mvn(datos, mvn_test = "hz")`.
3. **Prueba de Royston:** Extensión multivariante de Shapiro–Wilk, basada en el producto de estadísticas univariantes aplicadas a componentes principales. Fácil de interpretar; el poder disminuye en alta dimensión. En R: `MVN::mvn(datos, mvn_test = "royston")`.

4. **Prueba de Doornik–Hansen:** Combina asimetría y curtosis en una prueba ómnibus, alternativa robusta a Mardia. En R: `MVN::mvn(datos, mvn_test = "dh")`.
5. **Métodos gráficos (Q–Q chi-cuadrado):** Graficar las distancias de Mahalanobis contra cuantiles de una chi-cuadrada; desviaciones de la recta indican no normalidad multivariante. En R: `multivariate_diagnostic_plot(data = datos, type = "qq")`.

Capítulo 6

Análisis de componentes principales

El Análisis de Componentes Principales (PCA) es una técnica potente para reducir la dimensionalidad preservando la mayor información posible (medida por la varianza). Transforma las variables originales en un nuevo conjunto de variables no correlacionadas llamadas componentes principales, ordenadas por la varianza que explican.

6.1. Entendiendo la varianza total

Sea Σ la matriz de covarianzas del conjunto. La *varianza total* se da por la traza:

$$\text{Total Variance} = \text{Tr } \Sigma = \sum_{i=1}^p \Sigma_{ii},$$

donde Σ_{ii} son las varianzas muestrales de cada variable. Usando la descomposición en valores/vectores propios (SVD) de Σ , existen matrices Q y Λ tales que:

$$\Sigma = Q\Lambda Q^T,$$

siendo Λ diagonal con los valores propios $\lambda_1, \dots, \lambda_p$. Entonces la varianza total puede escribirse como:

$$\text{Total Variance} = \text{Tr } \Sigma = \text{Tr } Q\Lambda Q^T = \text{Tr } \Lambda = \lambda_1 + \dots + \lambda_p.$$

Las columnas de Q son las *direcciones de los componentes principales*; los valores de Λ indican cuánta varianza explica cada componente. El primero (mayor valor propio) capta la mayor varianza, el segundo la siguiente, y así sucesivamente. Estos componentes inducen un nuevo sistema de coordenadas; cada punto puede representarse por sus *puntuaciones* de componentes, relacionadas con las variables por:

$$X = Q(PC) + \bar{x}$$

y

$$PC = Q^T(X - \bar{x}) \tag{6.1}$$

Aquí X es el vector en coordenadas originales, PC el de puntuaciones, y $\bar{x}$ el vector de medias.

Para ilustrar estas ideas, consideramos el conjunto de la figura:

sample_data_plot

Datos de muestra

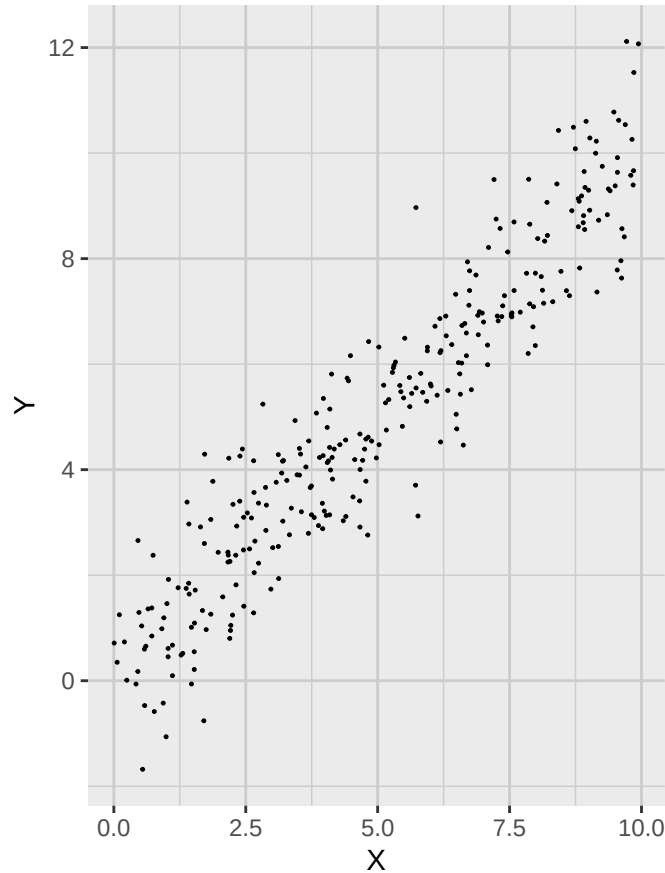


Figura 6.1: Datos de muestra con dos variables altamente correlacionadas.

Este conjunto consta de 300 observaciones de dos variables, X y Y : la primera es uniforme en $[0, 10]$ y la segunda es $Y = X + \epsilon$, con ϵ normal de media 0 y desviación 1. La matriz de covarianzas es:

```
Sigma <- cov(sample_data_tbl)
```

$$\Sigma = \begin{bmatrix} 7.874751 & 7.7745959 \\ 7.7745959 & 8.6510962 \end{bmatrix}$$

La varianza total es:

$$\text{Total Variance} = \text{Tr } \Sigma = \Sigma_{11} + \Sigma_{22} = 16.5258472$$

Consideramos ahora la descomposición espectral de Σ para obtener:


```
# Compute singular vectors and values
eig <- eigen(Sigma)
vectors <- eig$vectors
values <- eig$values
```

$$Q = \begin{bmatrix} 0.689251 & -0.7245227 \\ 0.7245227 & 0.689251 \end{bmatrix}$$

y

$$\Lambda = \begin{bmatrix} 16.0472039 & 0 \\ 0 & 0.4786433 \end{bmatrix}$$

Las direcciones de componentes principales son las columnas de Q y la varianza explicada por cada componente es el valor propio correspondiente. En particular, como el vector de medias es:

```
sample_mean_vector <- colMeans(sample_data_tbl)
```

$$\bar{x} = 4.9961185$$

$$\bar{y} = 5.019733$$

tenemos que la relación entre variables originales y coordenadas de componentes principales es:

$$X = 0.689251PC_1 + -0.7245227PC_2 + 4.9961185$$

$$Y = 0.7245227PC_1 + 0.689251PC_2 + 5.019733$$

y

$$PC_1 = 0.689251(X - 4.9961185) + 0.7245227(Y - 5.019733)$$

$$PC_2 = -0.7245227(X - 4.9961185) + 0.689251(Y - 5.019733)$$

Con estas fórmulas, obtenemos la proyección de los datos sobre el primer y segundo componente. Los resultados se muestran a continuación:

```
sample_data_matrix <- as.matrix(sample_data_tbl)

# Center the data
centered_sample_data <- scale(sample_data_matrix, center = TRUE, scale = FALSE)

# SVD
svd_result <- svd(centered_sample_data)
v1 <- svd_result$v[, 1]
v2 <- svd_result$v[, 2]

# Projection onto PC, in centered coordinates
proj_PC1 <- (centered_sample_data %*% v1) %*% t(v1)
proj_PC2 <- (centered_sample_data %*% v2) %*% t(v2)
```

```
# Shift back to original location
proj_coords1 <- proj_PC1 + attr(centered_sample_data, "scaled:center")
proj_coords2 <- proj_PC2 + attr(centered_sample_data, "scaled:center")

# Provide unique names before converting to tibble
colnames(proj_coords1) <- c("x_proj1", "y_proj1")
colnames(proj_coords2) <- c("x_proj2", "y_proj2")

proj_coords1_tbl <- as_tibble(proj_coords1)
proj_coords2_tbl <- as_tibble(proj_coords2)

# Bind with original data
compare_proj1_tbl <- bind_cols(sample_data_tbl, proj_coords1_tbl)
compare_proj2_tbl <- bind_cols(sample_data_tbl, proj_coords2_tbl)

# Generate plot corresponding to the first projection
compare_plot1 <- ggplot(compare_proj1_tbl) +
  geom_point(
    aes(x = x, y = y),
    color = "black",
    size = 0.2,
    alpha = 1
  ) +
  geom_point(
    aes(x = x_proj1, y = y_proj1),
    color = c_pal("C red"),
    size = 0.2,
    alpha = 1
  ) +
  labs(
    title = "Proyección en el primer componente principal",
    x = "X",
    y = "Y"
  ) +
  coord_fixed() +
  theme_krul()

# Generate plot corresponding to the second projection
compare_plot2 <- ggplot(compare_proj2_tbl) +
  geom_point(
    aes(x = x, y = y),
    color = "black",
    size = 0.2,
    alpha = 1
  )
```

```
) +  
geom_point(  
  aes(x = x_proj2, y = y_proj2),  
  color = c_pal("C red"),  
  size = 0.2,  
  alpha = 1  
) +  
labs(  
  title = "Proyección en el segundo componente principal",  
  x = "X",  
  y = "Y"  
) +  
coord_fixed() +  
theme_krul()  
  
# Combine plots  
compare_plot1 + compare_plot2 +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Proyecciones de componentes principales",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
)
```

Proyecciones de componentes principales

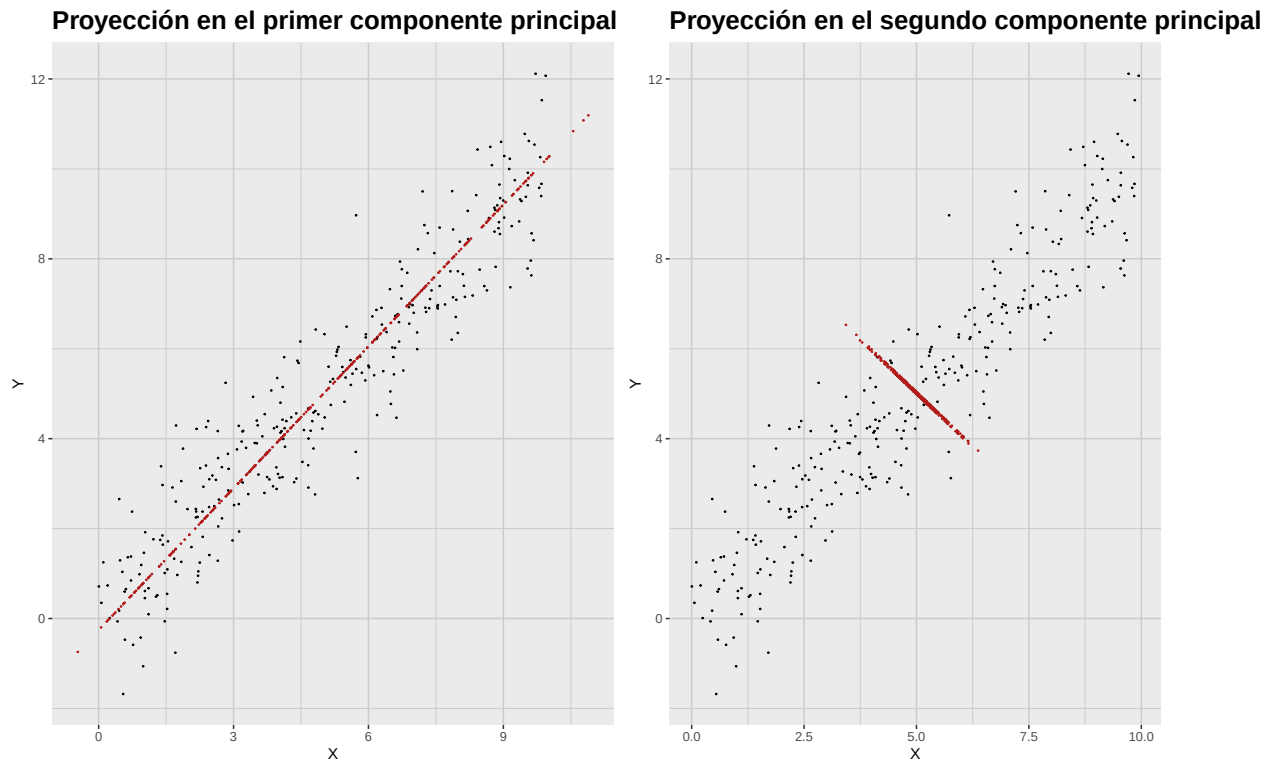


Figura 6.2: Proyecciones sobre las direcciones de los componentes principales. Se pierde mucha más información al proyectar sobre el segundo componente.

6.2. Varianza acumulada y gráficos scree

Como se indicó, el PCA se usa para reducción de dimensión. Buscamos reducir variables reteniendo la mayor información. Para decidir cuántos componentes conservar, examinamos la varianza explicada por cada uno. Es común usar un *gráfico scree*, que muestra valores propios en y y el índice del componente en x ; ayuda a identificar el “codo”. Otro gráfico útil es el de *varianza acumulada*, que muestra la proporción acumulada explicada. La proporción explicada por el componente k es:

$$\frac{\lambda_k}{\sum_{i=1}^p \lambda_i}$$

Este gráfico ayuda a determinar cuántos componentes se requieren para explicar, por ejemplo, 70 % o 90 % de la varianza total.

Para ilustrar, usaremos el conocido conjunto “iris”, con cuatro mediciones (largo/ancho de sépalo y pétalo) para tres especies (setosa, versicolor, virginica). Realizaremos PCA y construiremos el scree para visualizar la varianza explicada por cada componente. Primero, cargamos los datos y calculamos covarianzas por especie:

```
# Load the iris dataset
data(iris)
```

```
iris_setosa_tbl <- as_tibble(iris) %>%
  filter(Species == "setosa") %>%
  select(-Species)

iris_versicolor_tbl <- as_tibble(iris) %>%
  filter(Species == "versicolor") %>%
  select(-Species)

iris_virginica_tbl <- as_tibble(iris) %>%
  filter(Species == "virginica") %>%
  select(-Species)

# Compute covariance matrix
cov_matrix_setosa <- cov(iris_setosa_tbl)
cov_matrix_versicolor <- cov(iris_versicolor_tbl)
cov_matrix_virginica <- cov(iris_virginica_tbl)

# Singular value decomposition
eig_setosa <- eigen(cov_matrix_setosa)
eig_versicolor <- eigen(cov_matrix_versicolor)
eig_virginica <- eigen(cov_matrix_virginica)
```

La matriz de covarianza de cada especie es:

$$\Sigma_{setosa} = \begin{bmatrix} 0.12 & 0.1 & 0.02 & 0.01 \\ 0.1 & 0.14 & 0.01 & 0.01 \\ 0.02 & 0.01 & 0.03 & 0.01 \\ 0.01 & 0.01 & 0.01 & 0.01 \end{bmatrix}$$

$$\Sigma_{versicolor} = \begin{bmatrix} 0.27 & 0.09 & 0.18 & 0.06 \\ 0.09 & 0.1 & 0.08 & 0.04 \\ 0.18 & 0.08 & 0.22 & 0.07 \\ 0.06 & 0.04 & 0.07 & 0.04 \end{bmatrix}$$

$$\Sigma_{virginica} = \begin{bmatrix} 0.4 & 0.09 & 0.3 & 0.05 \\ 0.09 & 0.1 & 0.07 & 0.05 \\ 0.3 & 0.07 & 0.3 & 0.05 \\ 0.05 & 0.05 & 0.05 & 0.08 \end{bmatrix}$$

Los valores singulares de cada especie son:

$$\Lambda_{setosa} = \begin{bmatrix} 0.24 & 0 & 0 & 0 \\ 0 & 0.04 & 0 & 0 \\ 0 & 0 & 0.03 & 0 \\ 0 & 0 & 0 & 0.01 \end{bmatrix}$$

$$\Lambda_{versicolor} = \begin{bmatrix} 0.49 & 0 & 0 & 0 \\ 0 & 0.07 & 0 & 0 \\ 0 & 0 & 0.05 & 0 \\ 0 & 0 & 0 & 0.01 \end{bmatrix}$$

$$\Lambda_{virginica} = \begin{bmatrix} 0.7 & 0 & 0 & 0 \\ 0 & 0.11 & 0 & 0 \\ 0 & 0 & 0.05 & 0 \\ 0 & 0 & 0 & 0.03 \end{bmatrix}$$

Las direcciones asociadas a las componentes principales son las columnas de las matrices

$$Q_{setosa} = \begin{bmatrix} -0.67 & 0.6 & 0.44 & -0.04 \\ -0.73 & -0.62 & -0.27 & -0.02 \\ -0.1 & 0.49 & -0.83 & -0.24 \\ -0.06 & 0.13 & -0.2 & 0.97 \end{bmatrix}$$

$$Q_{versicolor} = \begin{bmatrix} -0.69 & 0.67 & -0.27 & 0.1 \\ -0.31 & -0.57 & -0.73 & -0.23 \\ -0.62 & -0.34 & 0.63 & -0.32 \\ -0.21 & -0.34 & 0.06 & 0.92 \end{bmatrix}$$

$$Q_{virginica} = \begin{bmatrix} -0.74 & -0.17 & -0.53 & 0.37 \\ -0.2 & 0.75 & -0.33 & -0.54 \\ -0.63 & -0.17 & 0.65 & -0.39 \\ -0.12 & 0.62 & 0.43 & 0.65 \end{bmatrix}$$

El diagrama de valores singulares (scree plot) y el de varianza acumulada para las tres especies se muestran en la figura siguiente:

```
# Scree plot data
scree_data <- tibble(
  Component = 1:4,
  Setosa = eig_setosa$values,
  Versicolor = eig_versicolor$values,
  Virginica = eig_virginica$values
) %>%
  pivot_longer(
    cols = -Component,
    names_to = "Species",
    values_to = "SingularValue"
  )
# Scree plot
```

```
scree_plot <- scree_data %>%
  ggplot(aes(x = Component, y = SingularValue, color = Species)) +
  geom_point(size = 3) +
  geom_line() +
  c_scale_color("C red", "C blue", "C green") +
  scale_x_continuous(breaks = 1:4) +
  labs(
    title = "Gráfico scree del conjunto Iris",
    x = "Componente principal",
    y = "Valor singular"
  ) +
  theme_krul() +
  theme(legend.position = "top")

# Cumulative variance data
cumulative_variance_data <- scree_data %>%
  group_by(Species) %>%
  arrange(Component) %>%
  mutate(CumulativeVariance = cumsum(SingularValue) / sum(SingularValue))

# Cumulative variance plot
cumulative_variance_plot <- cumulative_variance_data %>%
  ggplot(aes(x = Component, y = CumulativeVariance, color = Species)) +
  geom_point(size = 3) +
  geom_line() +
  c_scale_color("C red", "C blue", "C green") +
  scale_x_continuous(breaks = 1:4) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(
    title = "Varianza acumulada explicada por los componentes principales",
    x = "Componente principal",
    y = "Varianza acumulada explicada"
  ) +
  theme_krul() +
  theme(legend.position = "top")

# Combine scree plot and cumulative variance plot
scree_plot + cumulative_variance_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Gráfico scree y varianza acumulada para el conjunto Iris",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

Gráfico scree y varianza acumulada para el conjunto Iris

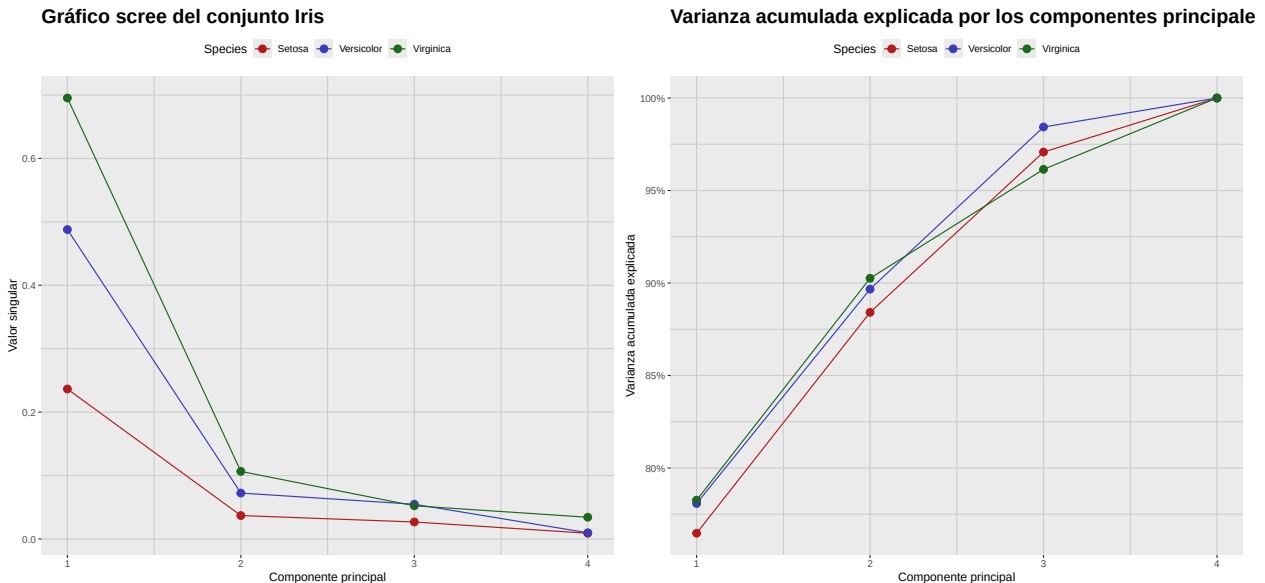


Figura 6.3: Análisis de Componentes Principales (ACP) del conjunto de datos Iris por especie. Para decidir cuántos componentes conservar, podemos usar el gráfico de sedimentación (scree plot, izquierda) y buscar el “codo” donde la varianza explicada se estabiliza. Alternativamente, podemos usar el gráfico de varianza acumulada (derecha) para determinar cuántos componentes se necesitan para explicar una cantidad deseada de varianza total, como 70 % o 90 %.

6.3. Cargas (loadings) y biplots

Recuerde que la ecuación (6.1) da la relación entre las variables originales y las puntuaciones de componentes principales. Con ella podemos calcular rápidamente las puntuaciones de cada observación. Los coeficientes de esta ecuación son las *cargas* de los componentes y cuantifican la contribución de cada variable. Un *biplot* muestra simultáneamente las puntuaciones de las observaciones y las cargas de las variables, permitiendo visualizar contribuciones y distribución en el espacio de los componentes.

Para ilustrar, crearemos biplots de los dos primeros componentes del conjunto “iris” por especie. Empezamos obteniendo las cargas correspondientes a cada especie, tomando las dos primeras

columnas de las matrices Q_{setosa} , $Q_{versicolor}$ y $Q_{virginica}$ mostradas arriba.

$$\text{Loadings}_{setosa} = \begin{bmatrix} -0.67 & 0.6 \\ -0.73 & -0.62 \\ -0.1 & 0.49 \\ -0.06 & 0.13 \end{bmatrix}$$

$$\text{Loadings}_{versicolor} = \begin{bmatrix} -0.69 & 0.67 \\ -0.31 & -0.57 \\ -0.62 & -0.34 \\ -0.21 & -0.34 \end{bmatrix}$$

$$\text{Loadings}_{virginica} = \begin{bmatrix} -0.74 & -0.17 \\ -0.2 & 0.75 \\ -0.63 & -0.17 \\ -0.12 & 0.62 \end{bmatrix}$$

Luego calculamos las puntuaciones multiplicando los datos centrados por estas matrices. Los biplots resultantes para cada especie se muestran a continuación:

```
# Function to prepare biplot data from precomputed singular vectors
prepare_biplot_from_eigen <- function(tbl, eig) {
  data_matrix <- as.matrix(tbl)
  centered_data_matrix <- scale(data_matrix, center = TRUE, scale = FALSE)

  # Scores: project observations onto PC1 and PC2
  scores <- centered_data_matrix %*% eig$vectors[, 1:2]
  colnames(scores) <- c("PC1", "PC2")
  scores_tbl <- as_tibble(scores) %>%
    mutate(Obs = row_number())

  # Loadings: first two singular vectors
  loadings <- eig$vectors[, 1:2]
  colnames(loadings) <- c("PC1", "PC2")
  loadings <- as_tibble(loadings) %>%
    mutate(Variable = colnames(tbl))

  list(scores = scores_tbl, loadings = loadings)
}

# Prepare data for each species
setosa_data <- prepare_biplot_from_eigen(
  iris_setosa_tbl,
  eig_setosa
)
```

```
versicolor_data <- prepare_biplot_from_eigen(
  iris_versicolor_tbl,
  eig_versicolor
)

virginica_data <- prepare_biplot_from_eigen(
  iris_virginica_tbl,
  eig_virginica
)

# Function to create biplot
create_biplot <- function(scores_tbl, loadings_tbl, species_name) {
  ggplot() +
    geom_point(
      data = scores_tbl,
      aes(x = PC1, y = PC2),
      color = c_pal("C red"),
      alpha = 1,
      size = 0.3
    ) +
    geom_segment(
      data = loadings_tbl,
      aes(
        x = 0,
        y = 0,
        xend = PC1 * max(scores_tbl$PC1),
        yend = PC2 * max(scores_tbl$PC2)
      ),
      arrow = arrow(length = unit(0.3, "cm")),
      color = c_pal("C blue"),
    ) +
    geom_text(
      data = loadings_tbl,
      aes(
        x = PC1 * max(scores_tbl$PC1) * 1.1,
        y = PC2 * max(scores_tbl$PC2) * 1.1,
        label = Variable
      ),
      color = "black",
      size = 5
    ) +
    labs(
      title = paste("Biplot para", species_name),
      x = "PC1",
      y = "PC2"
    )
}
```

```
    ) +  
    theme_minimal()  
}  
  
# Create biplots  
biplot_setosa <- create_biplot(  
  setosa_data$scores,  
  setosa_data$loadings,  
  "Setosa"  
)  
  
biplot_versicolor <- create_biplot(  
  versicolor_data$scores,  
  versicolor_data$loadings,  
  "Versicolor"  
)  
  
biplot_virginica <- create_biplot(  
  virginica_data$scores,  
  virginica_data$loadings,  
  "Virginica"  
)  
  
# Combine biplots  
biplot_setosa + biplot_versicolor + biplot_virginica +  
  plot_layout(ncol = 3) +  
  plot_annotation(  
    title = "Biplots de especies de Iris (PC1 vs PC2)",  
    theme = theme(  
      plot.title = element_text(size = 24, face = "bold")  
    )  
  )  
)
```

Biplots de especies de Iris (PC1 vs PC2)

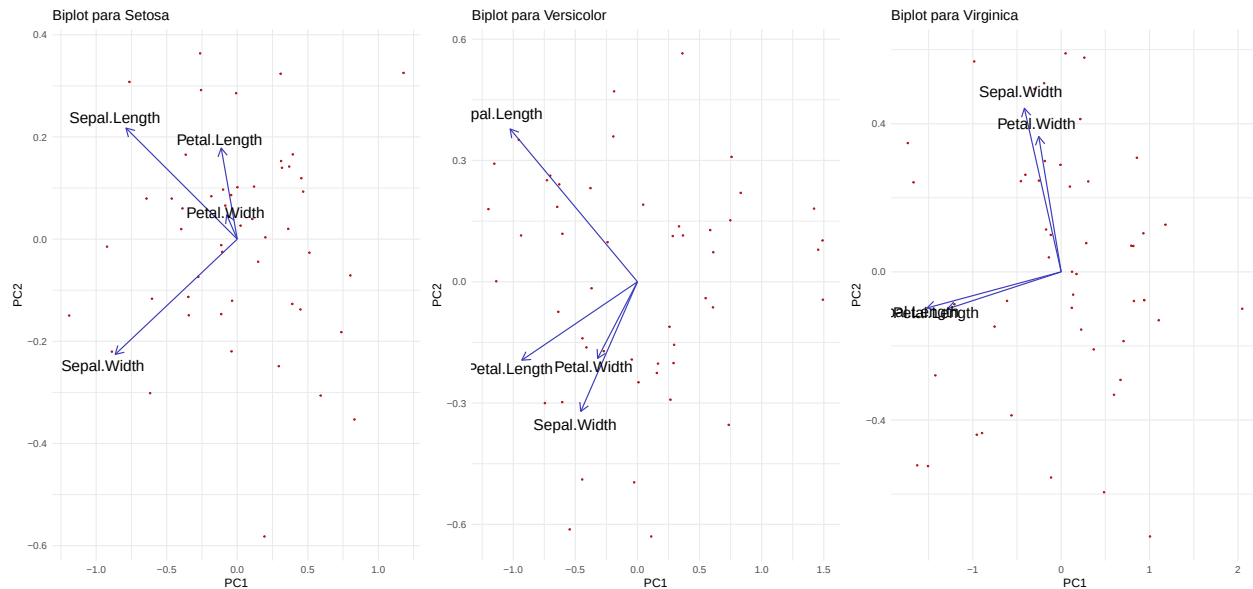


Figura 6.4: Cargas y puntuaciones para los dos primeros componentes principales del conjunto Iris por especie.

6.4. Resumen del proceso de PCA

En general, al realizar PCA se recomienda seguir estos pasos:

1. **Estandarice los datos (si es necesario).** No siempre es obvio si conviene estandarizar. Algunas pautas:
 - Si las variables están en escalas distintas (p.ej., cm y kg), suele recomendarse estandarizar.
 - Si están en la misma escala y con varianzas similares, podría no ser necesario.
 - Si las varianzas son muy diferentes, estandarizar ayuda a dar peso similar a todas.
2. **Calcule la matriz de covarianzas.**
 - La matriz captura cómo varían conjuntamente las variables.
 - Su tamaño es $p \times p$, donde p es el número de variables.
3. **Obtenga la descomposición en valores/vectores propios (SVD).**
 - Los autovalores miden la varianza explicada por cada componente.
 - Los autovectores dan las direcciones (combinaciones lineales) de los componentes.
4. **Ordene los componentes.**
 - Ordene los valores propios de mayor a menor.
 - El vector correspondiente al mayor valor propio es el primer componente.

5. **Decida cuántos componentes conservar.**

- *Criterio de Kaiser*: conservar valores propios > 1 (para matriz de correlación).
- *Gráfico scree*: buscar el “codo” donde se estabiliza la varianza explicada.
- *Varianza acumulada*: conservar componentes suficientes para 70–90 %.

6. **Obtenga puntuaciones y cargas.**

- Proyecte los datos originales sobre los componentes seleccionados.
- Use la fórmula: $PC = Q^T(X - \bar{x})$.

7. **Interprete los componentes.**

- Examine las *cargas* para ver qué variables contribuyen más.
- Valores absolutos grandes indican mayor influencia.
- Use biplots para visualizar puntuaciones y cargas conjuntamente.
- Aplique reducción de dimensión antes de regresión, clustering u otros análisis.
- Reduzca ruido descartando componentes de baja varianza.

Parte III

Análisis Factorial

Capítulo 7

Análisis factorial: modelos teóricos

El *análisis factorial* es una técnica estadística ampliamente usada para identificar patrones ocultos o estructuras latentes detrás de grandes conjuntos de variables observadas. Busca reducir la complejidad agrupando variables relacionadas en un menor número de *factores* subyacentes, más fáciles de interpretar.

Algunas áreas de aplicación comunes incluyen:

- **Psicología y ciencias sociales:** para descubrir rasgos latentes como la inteligencia, la ansiedad, dimensiones de la personalidad o niveles de satisfacción, a partir de respuestas a cuestionarios o encuestas.
- **Mercadotecnia e investigación del consumidor:** para identificar motivos de compra, percepciones de marca o segmentos de mercado a partir de datos de comportamiento de los clientes.
- **Finanzas y economía:** para revelar determinantes ocultos de los rendimientos bursátiles, indicadores macroeconómicos o factores de riesgo.
- **Educación:** para analizar reactivos de examen y determinar si miden habilidades o capacidades comunes.
- **Ciencias biológicas y de la salud:** para encontrar estructuras latentes en datos genéticos, listas de síntomas o resultados reportados por pacientes.

En la práctica se usa cuando se sospecha que unas cuantas dimensiones no observadas explican las correlaciones entre muchas variables. Es especialmente útil para construir escalas de medición, reducir datos y entender la estructura de conjuntos complejos.

7.1. Planteamiento del análisis factorial

El contexto teórico del análisis factorial es una relación de la forma

$$X = \mu + LF + \epsilon$$

donde:

- X es un vector aleatorio de p variables observadas (indicadores).
- μ es un vector constante de dimensión p .
- L es una matriz constante $p \times m$ conocida como la matriz de *cargas factoriales*.
- $F \sim \mathcal{N}(0, I_m)$ es un vector de m factores latentes (variables no observadas) que se asumen distribuidos normalmente con media cero y matriz de covarianzas identidad.
- $\epsilon \sim \mathcal{N}(0, \Psi)$ es un vector de errores únicos (varianzas específicas) que también se asumen distribuidos normalmente con media cero y matriz de covarianzas diagonal $\Psi = \text{diag}(\sigma_1^2, \dots, \sigma_p^2)$.
- Se asume que los factores F y los errores ϵ son independientes; en particular, $\text{Cov}[F, \epsilon] = 0$.

Como consecuencia de estos supuestos, se tiene

$$\mathbb{E}[X] = \mathbb{E}[\mu + LF + \epsilon] = \mathbb{E}[\mu] + L\mathbb{E}[F] + \mathbb{E}[\epsilon] = \mu$$

y

$$\begin{aligned} \text{Var}[X] &= \text{Var}[LF + \epsilon] \\ &= \mathbb{E}[(LF + \epsilon)(LF + \epsilon)^T] \\ &= \mathbb{E}[LFF^T L^T] + \mathbb{E}[\epsilon\epsilon^T] + \mathbb{E}[LF\epsilon^T] + \mathbb{E}[\epsilon F^T L^T] \\ &= L\mathbb{E}[FF^T]L^T + \mathbb{E}[\epsilon\epsilon^T] + L\mathbb{E}[F\epsilon^T] + \mathbb{E}[\epsilon F^T]L^T \\ &= LI_m L^T + \Psi + L0 + 0L^T \\ &= LL^T + \Psi \end{aligned}$$

Además, como X es combinación lineal de normales, se sigue que

$$X \sim \mathcal{N}(\mu, LL^T + \Psi)$$

7.2. Descomposición de varianza

La descomposición $\text{Var}[X] = LL^T + \Psi$ muestra que la varianza total de las variables observadas puede separarse en dos componentes:

- **Varianza común** (LL^T): explicada por los factores comunes F , capta la variabilidad compartida atribuible a la estructura latente.
- **Varianza única** (Ψ): específica de cada variable observada; incluye errores de medición y variabilidad propia.

En particular, la varianza de cada variable observada X_i puede expresarse como

$$\text{Var}[X_i] = \sum_{j=1}^m L_{ij}^2 + \sigma_i^2$$

donde $h_i^2 = \sum_{j=1}^m L_{ij}^2$ es la *comunalidad* de X_i (parte explicada por factores comunes) y σ_i^2 la varianza única.

7.3. Estimación de puntuaciones factoriales

Suponga el modelo

$$X = \mu + LF + \epsilon$$

con las mismas condiciones. Entonces la distribución conjunta de X y F es

$$\begin{bmatrix} X \\ F \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu \\ 0 \end{bmatrix}, \begin{bmatrix} LL^T + \Psi & L \\ L^T & I_m \end{bmatrix} \right)$$

Nótese que, en particular,

$$\text{Cov}[X, F] = \mathbb{E}[(X - \mu)(F - 0)^T] = \mathbb{E}[(LF + \epsilon)F^T] = L\mathbb{E}[FF^T] + \mathbb{E}[\epsilon F^T] = LI_m + 0 = L$$

Usando propiedades de la normal multivariante, se obtiene la distribución condicional de F dado X , que también es normal con media y covarianza

$$[F|X = x] \sim \mathcal{N} \left(L^T(LL^T + \Psi)^{-1}(x - \mu), I_m - L^T(LL^T + \Psi)^{-1}L \right)$$

La media de esta distribución condicional,

$$\mathbb{E}[F|X = x] = L^T(LL^T + \Psi)^{-1}(x - \mu)$$

se conoce como la *puntuación factorial* para la observación $X = x$. Proporciona una estimación de los factores latentes a partir de las variables observadas y las cargas. La matriz de covarianzas

$$I_m - L^T(LL^T + \Psi)^{-1}L$$

representa la incertidumbre asociada a las puntuaciones estimadas.

Comentario 7.3.1. Nótese que, si

$$f = \mathbb{E}[F|X = x] = L^T(LL^T + \Psi)^{-1}(x - \mu)$$

entonces, en general, el *residuo*

$$r(x) = x - \mu - Lf$$

es distinto de cero. De hecho, como función de X , el residuo tiene distribución

$$r(X) \sim \mathcal{N} \left(0, L \left(I_m + L^T \Psi^{-1} L \right)^{-1} L^T + \Psi \right)$$

7.4. Rotación de factores

Un aspecto importante del análisis factorial es que los factores no están definidos de manera única. Si F es un vector de factores, cualquier transformación ortogonal $F^* = QF$ (con $Q^T Q = I$) produce un nuevo conjunto de factores que explica la misma varianza. En particular,

$$\text{Var}[F^*] = \text{Var}[QF] = Q \text{Var}[F] Q^T = Q I_m Q^T = I_m$$

Las cargas correspondientes se transforman como $L^* = LQ^T$. En otras palabras, si

$$X = \mu + LF + \epsilon$$

es un modelo factorial válido, entonces también lo es

$$X = \mu + L^*F^* + \epsilon = \mu + LQ^TQF + \epsilon = \mu + LF + \epsilon$$

Esto implica que la solución no es única y distintas rotaciones pueden conducir a interpretaciones diferentes. En el siguiente capítulo exploraremos métodos prácticos y técnicas de rotación para lograr soluciones más interpretables.

Capítulo 8

Análisis factorial: métodos prácticos

Recordemos del capítulo anterior que el modelo clásico de análisis factorial es

$$X = \mu + LF + \epsilon$$

donde:

- X es un vector aleatorio de p variables observadas (indicadores).
- μ es un vector constante de dimensión p .
- L es una matriz constante $p \times m$ conocida como la matriz de *cargas factoriales*.
- $F \sim \mathcal{N}(0, I_m)$ es un vector de m factores latentes (variables no observadas) que se asumen distribuidos normalmente con media cero y matriz de covarianzas identidad.
- $\epsilon \sim \mathcal{N}(0, \Psi)$ es un vector de errores únicos (varianzas específicas) que también se asumen distribuidos normalmente con media cero y matriz de covarianzas diagonal $\Psi = \text{diag}(\sigma_1^2, \dots, \sigma_p^2)$.
- Se asume que los factores F y los errores ϵ son independientes; en particular, $\text{Cov}[F, \epsilon] = 0$.

Nótese que, como $\epsilon \sim \mathcal{N}(0, \Psi)$, este modelo queda completamente determinado por μ , L y Ψ . En este capítulo discutiremos métodos prácticos para estimar estos parámetros a partir de datos y para obtener las puntuaciones factoriales de F una vez estimado el modelo.

8.1. Evaluación de la factorizabilidad

El primer paso es evaluar si los datos son adecuados para AF; es decir, si las variables observadas exhiben correlaciones suficientes que justifiquen un modelo factorial. Varias pruebas y medidas pueden emplearse:

1. **Revisión de la matriz de correlaciones:** inspección visual en busca de patrones altos (p.ej., > 0.3) que sugieran factores comunes.
2. **Prueba de esfericidad de Bartlett:** contrasta identidad; un resultado significativo ($p < 0.05$) sugiere correlación suficiente.

3. **Kaiser–Meyer–Olkin (KMO):** mide la proporción de varianza atribuible a factores comunes; valores cercanos a 1 indican utilidad (típicamente > 0.6 es aceptable).

8.2. Decidir el número de factores

Determinar cuántos factores retener es crucial y puede afectar los resultados e interpretaciones. Algunos métodos:

1. **Criterio de Kaiser:** retener valores singulares > 1 .
2. **Gráfico scree:** buscar el “codo”.
3. **Análisis paralelo:** comparar con datos aleatorios del mismo tamaño.
4. **Índices de ajuste:** (RMSEA, TLI) para evaluar modelos con distinto número de factores.

8.3. Extracción de factores

Una vez decidido el número de factores, el siguiente paso es extraerlos. Métodos disponibles:

1. **Ejes principales (PAF):** se centra en la varianza compartida; estima communalidades y extrae factores desde la matriz reducida.
2. **Máxima verosimilitud (ML):** estima cargas maximizando la verosimilitud; permite pruebas e índices de ajuste.

8.4. Rotación de factores

Tras extraer factores, suele ser útil rotarlos para lograr una estructura más simple e interpretable. La rotación puede ser ortogonal (sin correlación) u oblicua (con correlación). Métodos comunes:

1. **Rotación Varimax:** Método de rotación ortogonal que maximiza la varianza de las cargas al cuadrado dentro de cada factor, logrando una estructura más simple donde cada variable carga fuertemente en un factor y débilmente en los demás.
2. **Rotación Quartimax:** Otro método de rotación ortogonal que simplifica las filas de la matriz de cargas, buscando que cada variable tenga cargas altas en la menor cantidad posible de factores.
3. **Rotación Equamax:** Método ortogonal híbrido que combina los objetivos de varimax y quartimax, equilibrando la simplificación de las filas y las columnas de la matriz de cargas.
4. **Rotación Promax:** Método de rotación oblicua que permite la correlación entre factores. Comienza con una rotación varimax y luego eleva las cargas a una potencia para obtener una estructura más simple.

8.5. Cálculo de puntuaciones factoriales

Una vez estimadas las cargas factoriales y rotados los factores (cuando aplica), el siguiente paso es calcular las puntuaciones factoriales para cada observación en el conjunto de datos. Estas puntuaciones representan los valores estimados de los factores latentes para cada persona a partir de sus puntuaciones observadas. Existen varios métodos para obtenerlas:

1. **Método de regresión:** estima puntuaciones al regredir las variables observadas sobre las cargas estimadas; es insesgado, pero puede no preservar la varianza original de los factores.
2. **Método de Bartlett:** calcula puntuaciones minimizando la varianza residual de las variables observadas; produce puntuaciones más fiables y con menor error de medición.
3. **Método de Anderson–Rubin:** genera puntuaciones no correlacionadas y estandarizadas; útil cuando se asume ortogonalidad de los factores.

8.6. Evaluación del ajuste global

Después de ajustar un modelo de análisis factorial, es importante evaluar qué tanto el modelo reproduce los datos observados. Diversas pruebas estadísticas e índices de ajuste permiten realizar esta evaluación:

1. **Prueba χ^2 de ajuste:** contrasta que la covarianza implícita del modelo coincida con la observada; un resultado no significativo ($p > 0.05$) sugiere buen ajuste.
2. **RMSEA:** discrepancia por grado de libertad entre covarianzas observada y modelada; valores < 0.06 indican buen ajuste.
3. **TLI (Tucker–Lewis):** compara el ajuste del modelo con un modelo nulo; valores > 0.95 indican buen ajuste.
4. **SRMR:** discrepancia media estandarizada entre correlaciones observadas y predichas; valores < 0.08 indican buen ajuste.

8.7. Visualización del modelo

Visualizar los resultados de un análisis factorial facilita la interpretación de los factores y el entendimiento de las relaciones entre variables. Algunas técnicas comunes son:

1. **Gráfico scree:** muestra los valores singulares en orden descendente y ayuda a decidir cuántos factores conviene retener.
2. **Diagrama de cargas factoriales:** un diagrama de dispersión de las cargas para apreciar cómo se distribuyen las variables en los distintos factores, especialmente después de rotar.
3. **Biplot:** combina en una sola figura las puntuaciones factoriales y las cargas, permitiendo visualizar simultáneamente a las observaciones y las variables en el espacio de factores.
4. **Mapa de calor de cargas:** representa la magnitud de las cargas factoriales mediante colores, resaltando patrones y posibles agrupamientos.

Comentario 8.7.1. Los diagramas de cargas y los biplots solo tienen sentido cuando hay 2 o 3 factores; de otro modo no se visualizan adecuadamente.

8.8. Ejemplo: análisis factorial del conjunto Holzinger-Swineford

Mostraremos ahora un ejemplo con el conjunto “HolzingerSwineford1939” del paquete “lavaan”. Incluye puntuaciones de 9 pruebas cognitivas aplicadas a 301 estudiantes de 7º y 8º. Las pruebas miden 3 habilidades latentes: visual/espacial, verbal y rapidez. En este ejemplo nos enfocaremos en x4 a x9 para ajustar un modelo de dos factores y visualizarlo en un biplot.

Una descripción detallada de cada prueba se presenta a continuación:

1. **Paragraph (x4)**: Comprensión lectora. Las y los estudiantes leen párrafos cortos y contestan preguntas sobre la idea principal, detalles e inferencias sencillas.
2. **Sentence (x5)**: Completar oraciones. Elegir la palabra o frase que mejor complete una oración para que sea clara y gramatical; enfatiza vocabulario en contexto y sintaxis.
3. **Word Meaning (x6)**: Vocabulario. Seleccionar el mejor sinónimo o definición para una palabra objetivo; refleja la amplitud del conocimiento léxico.
4. **Addition (x7)**: Fluidez aritmética cronometrada. Resolver rápidamente muchas sumas sencillas con precisión; captura velocidad de procesamiento con una carga mínima de razonamiento.
5. **Counting Dots (x8)**: Enumeración visual rápida. Contar con rapidez pequeños grupos de puntos a lo largo de muchos reactivos; mide el barrido visual y la precisión bajo presión de tiempo.
6. **Straight–Curved Capitals (x9)**: Velocidad de discriminación perceptual. Revisar letras mayúsculas y marcar aquellas con solo trazos rectos (versus cualquier trazo curvo) bajo límites estrictos de tiempo.

Para realizar este análisis, comenzaremos cargando las bibliotecas necesarias y el conjunto de datos:

```
# Load required libraries
library(lavaan)
library(psych)
library(scales)

# Load the Holzinger-Swineford1939 dataset
data(HolzingerSwineford1939)

# Select the relevant tests
hs_data <- as_tibble(HolzingerSwineford1939) %>%
  select("x4", "x5", "x6", "x7", "x8", "x9")
```

8.8.1. Evaluar la factorizabilidad

Iniciamos el proceso evaluando la factorizabilidad para x4–x9. Primero reunimos la información requerida:


```
# Correlation matrix (pairwise complete observations)
Sigma <- cor(hs_data)

# Bartlett's test and KMO measure
n_eff <- nrow(hs_data)
bart <- cortest.bartlett(Sigma, n = n_eff)
kmo <- KMO(Sigma)
```

Mostramos ahora el mapa de calor de correlaciones:

```
corr_df <- Sigma %>%
  as_tibble(rownames = "Var1") %>%
  pivot_longer(-Var1, names_to = "Var2", values_to = "r")

# Preserve matrix order on axes
corr_df$Var1 <- factor(corr_df$Var1, levels = colnames(Sigma))
corr_df$Var2 <- factor(corr_df$Var2, levels = colnames(Sigma))

ggplot(corr_df, aes(x = Var2, y = Var1, fill = r)) +
  geom_tile(color = "black") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 3) +
  scale_fill_gradient2(
    low = c_pal("C blue"), mid = "white", high = c_pal("C red"),
    midpoint = 0, limits = c(-1, 1)
  ) +
  coord_fixed() +
  labs(
    title = "Mapa de calor de correlaciones (x4-x9)",
    x = "", y = "", fill = "Correlación"
  ) +
  theme(
    plot.title = element_text(size = 18, face = "bold"),
    panel.background = element_blank(),
    panel.grid = element_blank(),
    axis.ticks = element_blank()
  )
```

Mapa de calor de correlaciones (x4–x9)

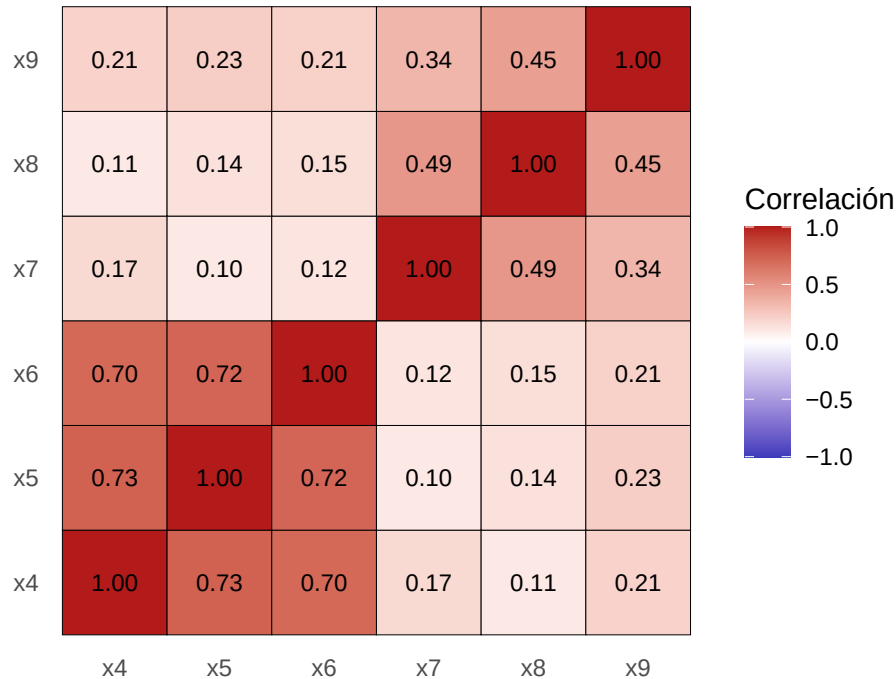


Figura 8.1: Mapa de calor de correlaciones para x4–x9. Se observan correlaciones fuertes entre las pruebas verbales (x4, x5, x6) y entre las de rapidez (x7, x8, x9), sugiriendo un modelo de dos factores.

Obsérvese la fuerte correlación entre pruebas verbales (x4, x5, x6) y entre rapidez (x7, x8, x9), pero no entre grupos. Esto sugiere que un modelo de dos factores es adecuado. Reportamos ahora Bartlett y KMO:

Prueba de esfericidad de Bartlett = 0

Medida Kaiser-Meyer-Olkin = 0.7308774

Como se observa, todas las pruebas indican que el AF es apropiado para este conjunto.

8.8.2. Decidir el número de factores

Usaremos un gráfico scree y uno de varianza acumulada para decidir cuántos factores retener. Como usamos la matriz de correlaciones, buscamos valores singulares > 1 (Kaiser) y el “codo” en el scree.

```
# Scree analysis from the correlation matrix
eig <- eigen(Sigma)

scree_df <- tibble(
  component = seq_along(eig$values),
  singular_value = eig$values
) %>%
```

```
mutate(
  cumulative = cumsum(singular_value) / sum(singular_value)
)

# Scree plot
scree_plot <- scree_df %>%
  ggplot(aes(x = component, y = singular_value)) +
  geom_point(size = 3, color = c_pal("C red")) +
  geom_line(color = c_pal("C blue")) +
  geom_hline(
    yintercept = 1,
    linetype = "dashed",
    color = c_pal("C green"),
    linewidth = 1.5
  ) +
  scale_x_continuous(breaks = scree_df$component) +
  labs(
    title = "Gráfico scree",
    x = "Componente principal",
    y = "Valor singular"
  ) +
  theme_krul()

# Cumulative variance plot
cum_plot <- ggplot(scree_df, aes(x = component, y = cumulative)) +
  geom_point(size = 3, color = c_pal("C red")) +
  geom_line(color = c_pal("C blue")) +
  scale_x_continuous(breaks = scree_df$component) +
  scale_y_continuous(
    labels = percent_format(accuracy = 1),
    limits = c(0, 1)
  ) +
  labs(
    title = "Varianza acumulada",
    x = "Componente principal",
    y = "Varianza acumulada explicada"
  ) +
  theme_krul()

# Combine scree plot and cumulative variance plot
scree_plot + cum_plot + plot_layout(ncol = 2) +
  plot_annotation(
    title = paste(
      "Gráfico scree y varianza acumulada para",
      "el conjunto Holzinger-Swineford"
    )
  )
```

```
),
  theme = theme(
    plot.title = element_text(
      size = 24,
      face = "bold"
    )
  )
)
```

Gráfico scree y varianza acumulada para el conjunto Holzinger–Swineford

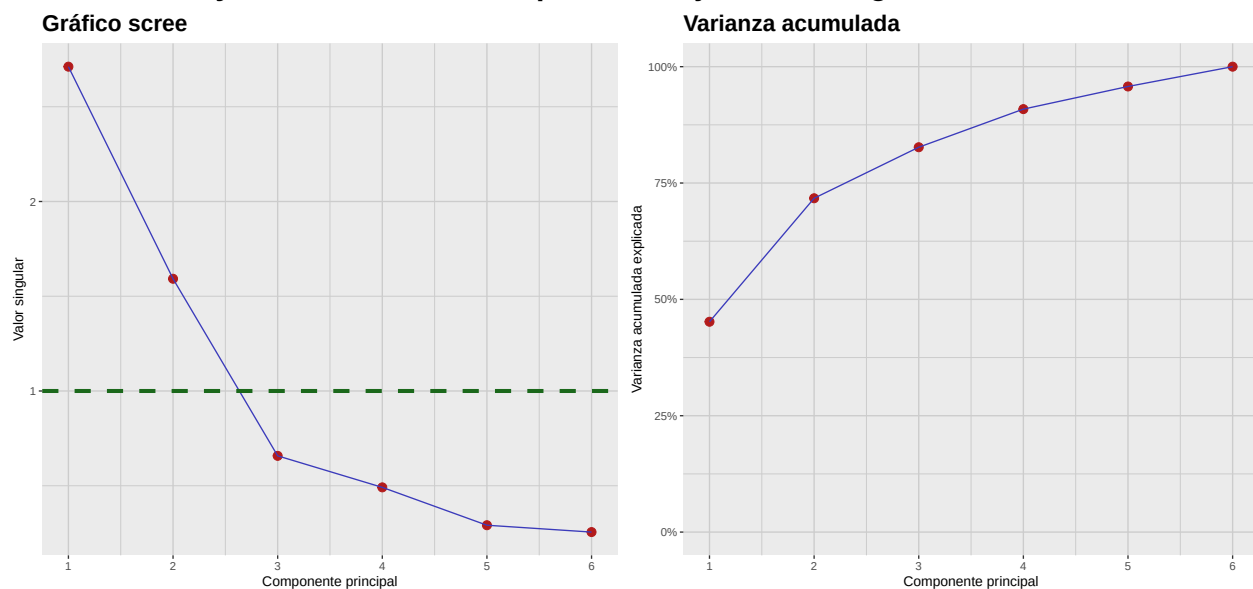


Figura 8.2: Gráfico scree y varianza acumulada para las pruebas x4–x9. Ambos gráficos sugieren que una solución de dos factores es adecuada para este conjunto de datos.

Nótese que tanto el gráfico scree como el de varianza acumulada sugieren que una solución de dos factores es adecuada para las seis pruebas consideradas.

8.8.3. Extracción, rotación y puntuación de factores

Aunque extraer, rotar y puntuar factores son pasos teóricos distintos, en la práctica suelen realizarse juntos con paquetes estadísticos. Aquí usaremos `psych::fa` para realizar los tres pasos de una vez: ajustar un modelo de 2 factores por máxima verosimilitud (ML), calcular puntuaciones por regresión y aplicar una rotación varimax para simplificar la estructura.

```
# Fit 2-factor FA using ML, regression scores and varimax rotation
fa_rot <- fa(
  r = hs_data,
  nfactors = 2,
  rotate = "varimax",
  fm = "ml",
```

```

    scores = "regression",
    use = "pairwise"
)

# Extract factor loadings into a tibble
loadings <- fa_rot$loadings[, 1:2] %>%
  unclass() %>%
  as_tibble(rownames = "Variable") %>%
  rename(F1 = 2, F2 = 3)
# Extract factor scores into a tibble
scores <- fa_rot$scores %>%
  as_tibble() %>%
  setNames(c("F1", "F2")) %>%
  mutate(Obs = row_number())

```

8.8.4. Evaluación del ajuste del modelo

Evaluamos qué tan bien la solución ML de 2 factores reproduce las correlaciones observadas. Para ello, reportamos χ^2 , RMSEA, CFI, TLI y RMSR.

```

# Extract standard fit indices from psych::fa object (ML)
chisq <- fa_rot$STATISTIC
df    <- fa_rot$dof
pval  <- fa_rot$PVAL
rmsea <- as.numeric(fa_rot$RMSEA[1])
tli   <- fa_rot$TLI
bic   <- fa_rot$BIC

L     <- as.matrix(fa_rot$loadings)
Phi   <- if (!is.null(fa_rot$Phi)) fa_rot$Phi else diag(ncol(L))
uni   <- if (!is.null(fa_rot$uniquenesses)) {
  fa_rot$uniquenesses
} else {
  1 - rowSums((L %*% Phi) * L)
}
Rhat  <- L %*% Phi %*% t(L) + diag(as.numeric(uni))

# Helper function to extract lower-triangle entries (off-diagonal)
lt_offdiag <- function(M) M[lower.tri(M, diag = FALSE)]

# Classic RMSR
res <- Sigma - Rhat
rmsr <- sqrt(mean(lt_offdiag(res)^2, na.rm = TRUE))

# Standardized RMSR (SRMR)
is_cov <- any(abs(diag(Sigma) - 1) > 1e-8)

```

```
S_obs_cor <- if (is_cov) stats::cov2cor(Sigma) else Sigma
S_hat_cor <- if (is_cov) stats::cov2cor(Rhat) else Rhat

res_cor <- S_obs_cor - S_hat_cor

# SRMR = sqrt( sum_{i<j}(res_ij^2) / sum_{i<j}(obs_ij^2) )
num <- sum(lt_offdiag(res_cor)^2, na.rm = TRUE)
den <- sum(lt_offdiag(S_obs_cor)^2, na.rm = TRUE)
srmr <- sqrt(num / den)
```

Los resultados son:

$$\begin{aligned}\text{Prueba } \chi^2 &= 0.1156716 \\ \text{RMSEA} &= 0.0531269 \\ \text{TLI} &= 0.9804573 \\ \text{RMSR} &= 0.0411927\end{aligned}$$

Todas las métricas indican que el modelo de 2 factores ajusta bien los datos.

8.8.5. Visualización del modelo

Finalmente, visualizamos el modelo con un biplot. También rotaremos manualmente los factores para ilustrar qué pasaría sin una rotación adecuada.

```
# Scaling factor for arrows (keep consistent across plots)
arrow_scale <- 3

# Base (unrotated) biplot
p_base <- ggplot() +
  geom_point(
    data = scores,
    aes(x = F1, y = F2),
    alpha = 1,
    color = c_pal("C red"),
    size = 0.5
  ) +
  geom_segment(
    data = loadings,
    aes(x = 0, y = 0, xend = F1 * arrow_scale, yend = F2 * arrow_scale),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C blue"),
    linewidth = 1.2
  ) +
  geom_text(
    data = loadings,
    aes(x = F1 * arrow_scale, y = F2 * arrow_scale, label = Variable),
```

```
    color = "black",
    hjust = -0.5
) +
labs(
  title = "Rotación Varimax",
  x = "Factor 1",
  y = "Factor 2"
) +
theme_krul()

# 45° rotation of scores and loadings
theta <- pi / 4
cos_t <- cos(theta)
sin_t <- sin(theta)

scores_rot <- scores %>%
  mutate(
    F1_rot = F1 * cos_t - F2 * sin_t,
    F2_rot = F1 * sin_t + F2 * cos_t
  )

loadings_rot <- loadings %>%
  mutate(
    F1_rot = F1 * cos_t - F2 * sin_t,
    F2_rot = F1 * sin_t + F2 * cos_t
  )

# Rotated biplot
p_rot <- ggplot() +
  geom_point(
    data = scores_rot,
    aes(x = F1_rot, y = F2_rot),
    alpha = 1,
    color = c_pal("C red"),
    size = 0.5
  ) +
  geom_segment(
    data = loadings_rot,
    aes(x = 0, y = 0, xend = F1_rot * arrow_scale, yend = F2_rot * arrow_scale),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C blue"),
    linewidth = 1.2
  ) +
  geom_text(
    data = loadings_rot,
```

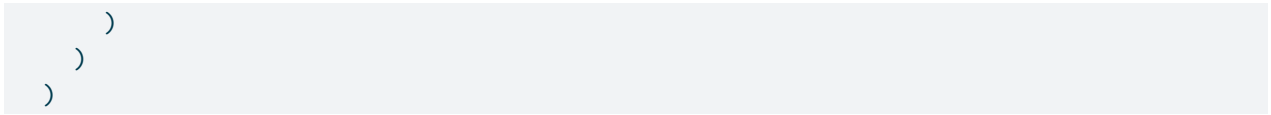
```
    aes(x = F1_rot * arrow_scale, y = F2_rot * arrow_scale, label = Variable),
    color = "black",
    hjust = -0.5
  ) +
  labs(
    title = "Rotación manual",
    x = "Factor 1 (rotado)",
    y = "Factor 2 (rotado)"
  ) +
  theme_krul()

# Optional: share axis limits for fair comparison
xr <- range(
  c(
    scores$F1,
    scores_rot$F1_rot,
    loadings$F1 * arrow_scale,
    loadings_rot$F1_rot * arrow_scale
  ),
  na.rm = TRUE
)

yr <- range(
  c(
    scores$F2,
    scores_rot$F2_rot,
    loadings$F2 * arrow_scale,
    loadings_rot$F2_rot * arrow_scale
  ),
  na.rm = TRUE
)

p_base <- p_base + coord_equal(xlim = xr, ylim = yr)
p_rot <- p_rot + coord_equal(xlim = xr, ylim = yr)

# Side-by-side using patchwork
p_base + p_rot + plot_layout(ncol = 2) +
  plot_annotation(
    title = paste(
      "Biplots del modelo de análisis factorial",
      "usando rotación Varimax y manual"
    ),
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

Biplots del modelo de análisis factorial usando rotación Varimax y manual

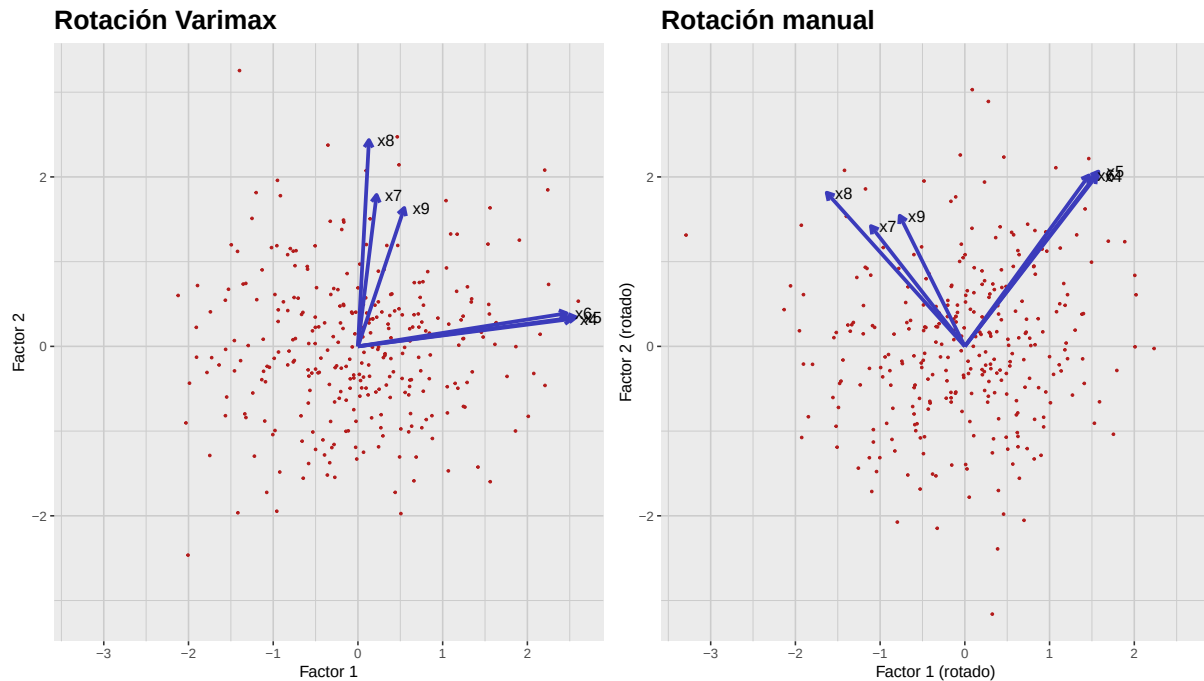


Figura 8.3: Biplots de la solución de 2 factores para x4–x9. Izquierda: rotación Varimax. Derecha: rotación manual 45°. La rotación Varimax produce una estructura más simple e interpretable: las pruebas verbales (x4, x5, x6) cargan alto en el Factor 1 y las de rapidez (x7, x8, x9) en el Factor 2.

Parte IV

Análisis de Conglomerados

Capítulo 9

Análisis de conglomerados jerárquicos

Clustering (agrupamiento) es una técnica de aprendizaje no supervisado que agrupa puntos de datos similares con base en sus características. A diferencia del aprendizaje supervisado, el agrupamiento no depende de etiquetas o categorías predefinidas; en su lugar, identifica patrones y estructuras dentro de los propios datos. Los métodos de agrupamiento suelen clasificarse en dos familias principales:

1. **Agrupamiento jerárquico:** Estos métodos construyen un *dendrograma* (también llamado *diagrama de árbol*) que representa la estructura taxonómica de los datos. Es especialmente útil para conjuntos de datos pequeños y cuando se desconoce el número de conglomerados.
2. **Agrupamiento no jerárquico:** Métodos como K-medias y DBSCAN particionan los datos en un número predefinido de conglomerados o con base en densidad. Generalmente escalan mejor para conjuntos de datos grandes.

En este capítulo nos enfocaremos en el agrupamiento jerárquico, dejando los métodos no jerárquicos para el siguiente.

9.1. Entendiendo los dendrogramas

Un dendrograma es un diagrama en forma de árbol que ilustra la estructura taxonómica de los datos de acuerdo con un algoritmo de agrupamiento jerárquico. Proporciona una representación visual de cómo se agrupan los puntos según sus similitudes. El eje vertical representa la distancia o disimilitud entre conglomerados, mientras que el eje horizontal representa los puntos individuales o los propios conglomerados. Al “cortar” el dendrograma a una altura específica, podemos determinar el número de conglomerados que deseamos considerar para el análisis.

Como ilustración, consideremos el diagrama de dispersión del conjunto de datos Iris (considerando únicamente las variables “Sepal.Length” y “Sepal.Width”), donde cada punto está coloreado según la especie de la flor:

```
data(iris)

iris_plot <- iris %>%
  ggplot(aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
```

```
geom_point(
  size = 0.8,
  alpha = 1
) +
c_scale_color("C green", "C blue", "C red") +
labs(
  title = "Conjunto de datos Iris: Sepal.Length vs Sepal.Width",
  x = "Sepal Length (cm)",
  y = "Sepal Width (cm)"
) +
theme_krul() +
theme(legend.position = "top")

iris_plot
```

Conjunto de datos Iris: Sepal.Length vs Sepal.

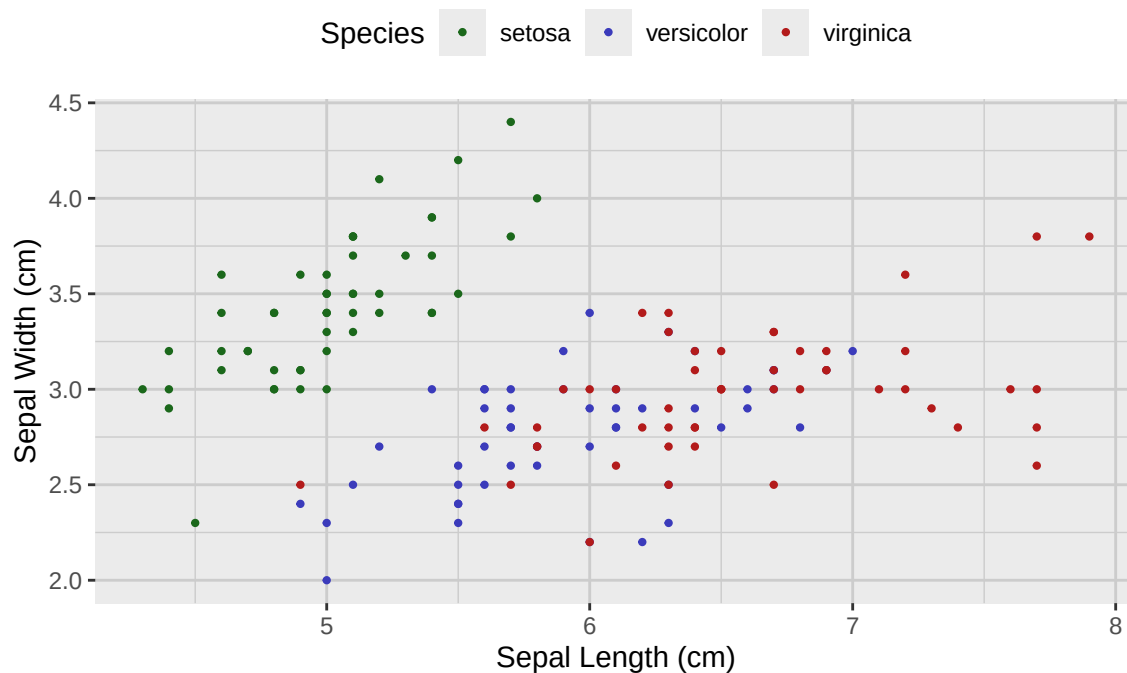


Figura 9.1: Diagrama de dispersión del conjunto de datos Iris, separado por especie y considerando únicamente las variables 'Sepal.Length' y 'Sepal.Width'.

Ahora consideraremos un dendrograma asociado a este conjunto y mostraremos cómo cortarlo a distintas alturas conduce a diferentes números de conglomerados:

```
# Use the built-in iris dataset only
iris_tbl <- as_tibble(iris)
```

```
k_values <- c(1, 3, 4, 6)

# Cluster on the chosen 2D projection to match the plot
standardized_sepal_matrix <- iris_tbl %>%
  select(Sepal.Length, Sepal.Width) %>%
  scale()

distances <- stats::dist(standardized_sepal_matrix, method = "euclidean")
sepal_hclust <- stats::hclust(distances, method = "ward.D2")

sepal_tbl <- iris_tbl %>%
  select(Sepal.Length, Sepal.Width) %>%
  mutate(.row_id = row_number())

# Build long data for facets by k
long_pts <- lapply(k_values, function(k) {
  cluster <- stats::cutree(sepal_hclust, k = k)
  tibble::tibble(
    .row_id = seq_along(cluster),
    cluster = factor(as.integer(cluster)),
    k = factor(paste0("k = ", k), levels = paste0("k = ", k_values))
  )
})

long_pts <- bind_rows(long_pts)
sepal_cluster_tbl <- left_join(long_pts, sepal_tbl, by = ".row_id")

# Polygons for each (k, cluster)
sepal_poly_tbl <- cluster_polygons(
  sepal_cluster_tbl,
  Sepal.Length, Sepal.Width,
  k, cluster
)

sepal_cluster_plot <- sepal_cluster_tbl %>%
  ggplot(aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(color = "black", alpha = 1, size = 0.8) +
  labs(
    title = "Clusters",
    x = "Sepal Length",
    y = "Sepal Width"
  ) +
  theme_krul() +
  facet_wrap(~k, ncol = 1, scales = "fixed")
```

```
sepal_cluster_plot <- sepal_cluster_plot +
  geom_path(
    data = sepal_poly_tbl,
    aes(
      x = Sepal.Length,
      y = Sepal.Width,
      group = interaction(k, cluster),
      color = cluster
    ),
    linewidth = 0.8,
    alpha = 0.5
  ) +
  c_scale_color(
    "C green",
    "C blue",
    "C red",
    "C magenta",
    "C yellow",
    "C cyan"
  ) +
  labs(color = "Cluster")

# Dendrogram geometry (replicated across facets)
sepal_dendro_data <- ggdendro::dendro_data(sepal_hclust, type = "rectangle")
k_fac <- factor(paste0("k = ", k_values), levels = paste0("k = ", k_values))

dendro_segments_faceted <- sepal_dendro_data$segments %>%
  mutate(.rep = 1) %>%
  tidyr::crossing(k = k_fac) %>%
  select(-.rep)

# One dashed cut line per k facet
dendro_cuts_tbl <- tibble::tibble(
  k = k_fac,
  y_cut = vapply(
    k_values,
    function(k) cut_dendrogram_height_for_k(sepal_hclust, k),
    numeric(1)
  )
)

dendro_faceted <- ggplot() +
  geom_segment(
    data = dendro_segments_faceted,
    aes(x = x, y = y, xend = xend, yend = yend),
```



```
    linewidth = 0.4
  ) +
  geom_hline(
    data = dendro_cuts_tbl,
    aes(yintercept = y_cut),
    linetype = "dashed",
    color = c_pal("C red")
  ) +
  labs(title = "Dendrogram cuts", x = NULL, y = "Height") +
  theme_krul() +
  theme(
    axis.text.x = element_blank(),
    axis.ticks.x = element_blank(),
    panel.grid = element_blank()
  ) +
  facet_wrap(~k, ncol = 1, scales = "fixed")

combined_plot <- (dendro_faceted | sepal_cluster_plot) +
  plot_layout(widths = c(1, 2)) +
  plot_annotation(
    title = "Agrupamiento jerárquico del conjunto de datos Iris.",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

combined_plot
```

Agrupamiento jerárquico del conjunto de datos Iris.

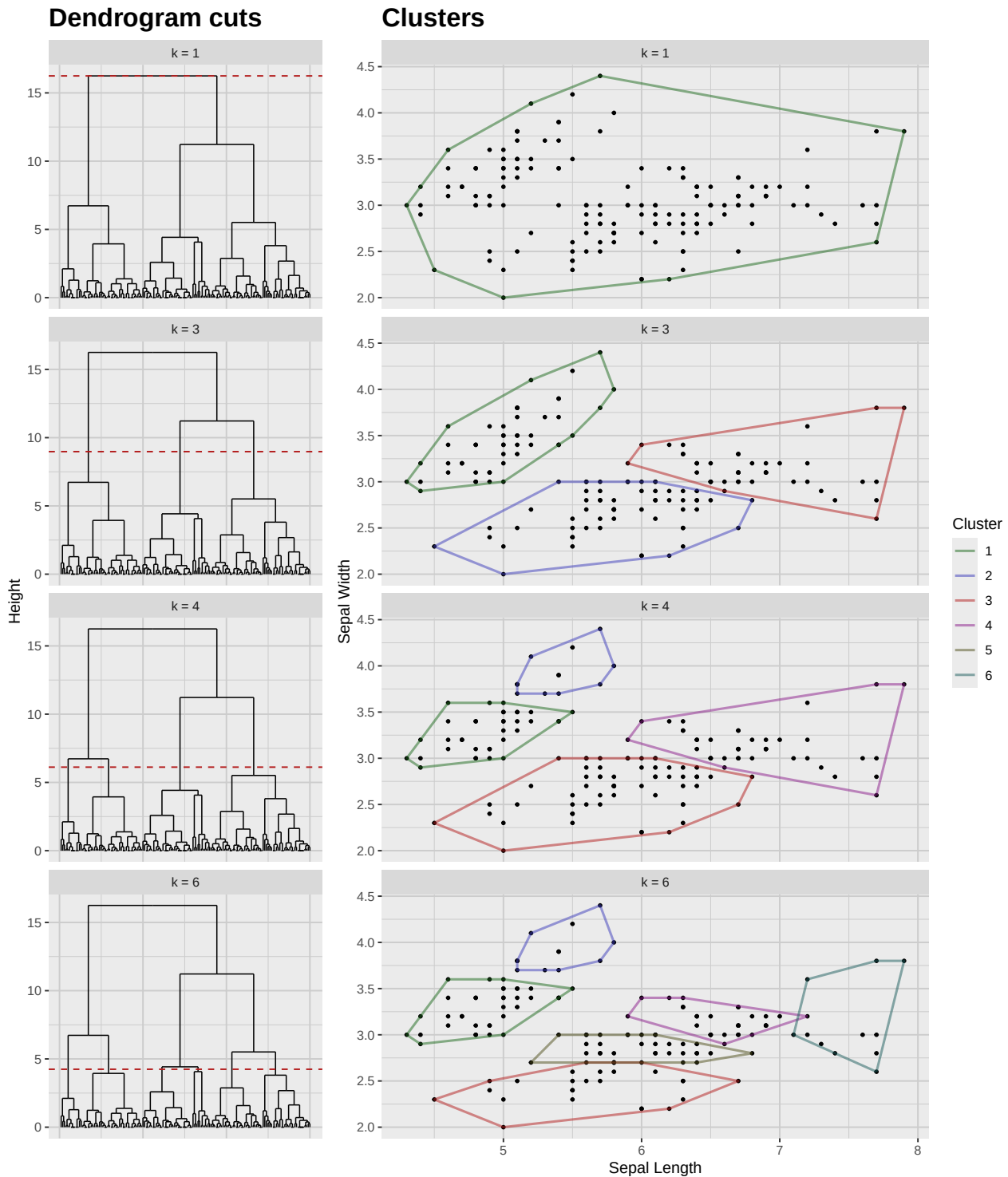


Figura 9.2: Agrupamiento jerárquico del conjunto de datos Iris considerando únicamente las variables 'Sepal.Length' y 'Sepal.Width'. Las líneas punteadas indican los puntos de corte para los números de clústeres dados ($k = 1, 3, 4, 6$).

9.2. Realización del agrupamiento jerárquico

Existen varios métodos para realizar agrupamiento jerárquico, pero los más comunes son:

- **Agrupamiento aglomerativo:** Enfoque ascendente en el que cada punto inicia como su propio conglomerado y, de manera iterativa, se fusionan parejas de conglomerados según su similitud hasta formar uno solo o hasta cumplir un criterio de parada.
- **Agrupamiento divisivo:** Enfoque descendente en el que todos los puntos empiezan en un único conglomerado y este se divide recursivamente en conglomerados más pequeños según su disimilitud hasta que cada punto queda solo o se cumple un criterio de parada.

Una vez elegido el método, es necesario definir una métrica de distancia para medir la similitud entre puntos. Algunas métricas comunes son:

- **Distancia euclidiana:** La métrica más utilizada; calcula la distancia en línea recta entre dos puntos en un espacio multidimensional.
- **Distancia de Mahalanobis:** Considera las correlaciones entre variables y es útil al tratar con datos multivariados.
- **Distancia de Canberra:** Sensible a pequeños cambios en los datos y útil cuando se trabaja con datos dispersos (escasos). La fórmula de la distancia de Canberra entre dos puntos p y q es:

$$d(p, q) = \sum_{i=1}^n \frac{|p_i - q_i|}{|p_i| + |q_i|}$$

donde n es el número de dimensiones, y p_i y q_i son las coordenadas de los puntos p y q , respectivamente.

- **Distancia Manhattan:** También conocida como norma L^1 o distancia de taxista; calcula la distancia entre dos puntos sumando las diferencias absolutas de sus coordenadas. Es particularmente útil en alta dimensión o cuando los datos tienen estructura en rejilla. La fórmula para dos puntos p y q es:

$$d(p, q) = \sum_{i=1}^n |p_i - q_i|$$

- **Distancia de Minkowski:** Generaliza las distancias Euclidiana y Manhattan, permitiendo distintos valores del parámetro p para controlar el cálculo. Para dos puntos p y q se define como:

$$d(p, q) = \left(\sum_{i=1}^n |p_i - q_i|^p \right)^{1/p}$$

Cuando $p = 1$, corresponde a la distancia Manhattan; cuando $p = 2$, a la distancia Euclidiana.

Finalmente, debemos elegir un criterio de enlace para determinar cómo se calcula la distancia entre conglomerados. Algunos criterios comunes son:

- **Enlace simple (single):** Define la distancia entre dos conglomerados como la distancia mínima entre cualquier par de puntos de cada conglomerado. Tiende a producir cadenas largas y es sensible al ruido y a valores atípicos.

- **Enlace completo (complete):** Define la distancia entre dos conglomerados como la distancia máxima entre cualquier par de puntos de cada conglomerado. Tiende a crear conglomerados compactos y aproximadamente esféricos, y es menos sensible al ruido y a atípicos que el enlace simple.
- **Enlace promedio (average):** Define la distancia entre dos conglomerados como la distancia promedio entre todos los pares de puntos de cada conglomerado. Proporciona un balance entre enlace simple y completo y es menos sensible al ruido y a atípicos.
- **Enlace de Ward:** Minimiza la varianza total intra-conglomerado al fusionar los conglomerados que ocasionan el menor incremento de varianza. Suele producir conglomerados compactos y aproximadamente esféricos, y es menos sensible al ruido y a atípicos que otros criterios de enlace.

Una vez elegidos el método, la métrica de distancia y el criterio de enlace, podemos realizar el agrupamiento jerárquico con la función `hclust` en R. Como se mostró en el ejemplo anterior, luego podemos visualizar los resultados con un dendrograma y cortarlo a una altura específica para determinar el número de conglomerados.

Capítulo 10

Agrupamiento no jerárquico

En el *agrupamiento no jerárquico* particionamos los datos en un número predeterminado de conglomerados. En este caso no obtenemos un dendrograma, sino un conjunto plano de conglomerados. Existen muchos métodos de agrupamiento no jerárquico, pero la mayoría pueden clasificarse como *métodos de particionado*, *métodos basados en densidad* o *métodos basados en modelos*.

10.1. Métodos de particionado

Estos métodos crean una partición de los datos en un conjunto de k conglomerados, donde k lo especifica el usuario. Se basan en optimizar alguna función objetivo que mide la calidad del agrupamiento. Los métodos de particionado más comunes son:

- **Agrupamiento k-means:** Este método busca particionar los datos en k conglomerados de forma que se minimice la suma de distancias cuadráticas entre cada punto y el centroide de su conglomerado asignado. El algoritmo asigna iterativamente los puntos al centroide más cercano y actualiza los centroides hasta converger.
- **Agrupamiento k-medoids:** Similar a k-means, pero en lugar de usar la media de los puntos como centroide, utiliza un punto real de los datos (el medoide) que minimiza la suma de distancias a los demás puntos del conglomerado. Esto hace a k-medoids más robusto al ruido y a valores atípicos.
- **CLARA (Clustering LARge Applications):** Extensión de k-medoids diseñada para conjuntos grandes; muestrea un subconjunto de datos y aplica k-medoids a esa muestra.
- **PAM (Partitioning Around Medoids):** Implementación específica de k-medoids que usa una estrategia voraz para encontrar los medoides y asignar puntos a conglomerados.

10.2. Métodos basados en densidad

Estos métodos identifican conglomerados con base en la densidad de puntos en el espacio de datos. Pueden encontrar conglomerados de formas arbitrarias y son robustos al ruido. Los métodos más comunes son:

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):** Define conglomerados como zonas de alta densidad separadas por regiones de baja densidad. Requiere dos parámetros: el radio ϵ que define la vecindad de un punto y el número mínimo de puntos requerido para formar una región densa. Los puntos que no pertenecen a ningún conglomerado se consideran ruido.
- **OPTICS (Ordering Points To Identify the Clustering Structure):** Extensión de DBSCAN que ordena los puntos según su densidad, permitiendo identificar conglomerados a distintos niveles de densidad.
- **Mean Shift:** Método que desplaza iterativamente cada punto hacia la media de los puntos en su vecindad, encontrando los modos de la distribución. Los conglomerados se forman alrededor de esos modos.

10.3. Métodos basados en modelos

Suponen que los datos provienen de una mezcla de distribuciones de probabilidad subyacentes y buscan identificar dichas distribuciones, asignando puntos a conglomerados según su verosimilitud de pertenencia. Los más comunes son:

- **Modelos de Mezclas Gaussianas (GMM):** Modelan los datos como mezcla de varias distribuciones gaussianas, cada una representando un conglomerado. Los parámetros se estiman mediante el algoritmo de Expectación–Maximización (EM) y los puntos se asignan según probabilidades posteriores.
- **Agrupamiento Bayesiano:** Utiliza inferencia bayesiana para estimar parámetros de las distribuciones subyacentes y asignar puntos a conglomerados. Puede incorporar conocimiento previo y manejar la incertidumbre del proceso de agrupamiento.
- **Análisis de Clases Latentes (LCA):** Usado para datos categóricos; asume que los datos provienen de una mezcla de clases latentes. Estima parámetros de las clases y asigna puntos según probabilidades posteriores.

10.4. Elección del método adecuado

La elección del método de agrupamiento no jerárquico depende de las características de los datos y de los objetivos del análisis. Algunas pautas:

- Si los datos son continuos y se espera que los conglomerados sean aproximadamente esféricos y de tamaño similar, k-means o k-medoids pueden ser apropiados.
- Si hay ruido o valores atípicos, o se esperan conglomerados de diferentes formas y tamaños, métodos basados en densidad como DBSCAN u OPTICS pueden ser más adecuados.
- Si se cree que los datos provienen de una mezcla de distribuciones subyacentes, métodos basados en modelos como GMM o el agrupamiento bayesiano pueden ser la mejor opción.
- Si los datos son categóricos, el Análisis de Clases Latentes (LCA) puede ser el método más apropiado.

- Si el conjunto es grande, considere métodos como CLARA, diseñados para manejar grandes volúmenes eficientemente.

10.5. Evaluación de los resultados de agrupamiento

Evaluar la calidad del agrupamiento es crucial para confirmar que el método elegido capturó la estructura subyacente. Algunas técnicas comunes son:

- **Validación interna:** Usa métricas basadas en los propios datos, sin referencia externa. Entre las más comunes:
 - *Índice de Silueta:* Mide qué tan similar es un punto a su propio conglomerado frente a otros. Valores altos indican conglomerados bien definidos.
 - *Índice de Davies–Bouldin:* Evalúa la razón de similitud promedio de cada conglomerado con su más similar. Valores bajos indican mejor agrupamiento.
 - *Índice de Calinski–Harabasz:* Mide la razón varianza entre–conglomerados vs. intra–conglomerados. Valores altos indican conglomerados mejor definidos.
- **Validación externa:** Compara contra etiquetas conocidas o verdad-terreno, si existen. Métricas comunes:
 - *Índice de Rand Ajustado (ARI):* Mide la similitud con la verdad-terreno, ajustando por azar. Valores altos indican mayor acuerdo.
 - *Información Mutua Normalizada (NMI):* Cuantifica la información compartida entre el agrupamiento y la verdad-terreno. Valores altos indican mayor acuerdo.
- **Análisis de estabilidad:** Evalúa la robustez aplicando el algoritmo a subconjuntos o añadiendo ruido. Resultados consistentes entre corridas indican estabilidad.
- **Inspección visual:** Diagramas de dispersión, mapas de calor o dendrogramas (si aplica) ayudan a entender la estructura y a detectar posibles problemas.

10.6. Ejemplo en R

Mostramos ahora cómo implementar distintos métodos de agrupamiento no jerárquico en R usando el conjunto `iris`. Iniciamos cargando las librerías necesarias.

```
library(cluster)
library(dbSCAN)
library(mclust)
library(krnlRutils)
```

Luego preparamos los datos y mostramos una figura comparando los resultados de distintos métodos de agrupamiento.

```
set.seed(123)

# Usar únicamente Sepal.Width y Sepal.Length
```

```
iris_tbl <- as_tibble(iris) %>%
  select(Sepal.Width, Sepal.Length)

iris_matrix <- as.matrix(iris_tbl)

# k-means (particionado)
iris_kmeans <- kmeans(iris_matrix, centers = 3, nstart = 50)$cluster

# PAM (k-medoids, particionado)
iris_pam <- pam(iris_matrix, k = 3)$clustering

# GMM (basado en modelos, mezclas gaussianas)
gmm <- Mclust(iris_matrix, G = 3)$classification

db_raw <- choose_dbscan(iris_matrix, min_pts = 5)
db <- assign_noise_to_nearest(iris_matrix, db_raw$cluster)

# Unir resultados para graficación facetada
labs <- list(
  `k-means` = iris_kmeans,
  PAM = iris_pam,
  GMM = gmm,
  DBSCAN = db
)

plot_df <- bind_rows(lapply(names(labs), function(m) {
  tibble(
    sepal_width = iris_tbl$Sepal.Width,
    sepal_length = iris_tbl$Sepal.Length,
    method = m,
    cluster = factor(labs[[m]])
  )
}))

ggplot(plot_df, aes(sepal_width, sepal_length, color = cluster)) +
  geom_point(size = 1.3, alpha = 1) +
  c_scale_color("C red", "C green", "C blue") +
  facet_wrap(~ method, ncol = 2) +
  labs(title = "Agrupamiento de Iris (sépalos)",
       x = "Ancho de sépalo", y = "Largo de sépalo") +
  theme_krul()
```


Agrupamiento de Iris (sépalos)

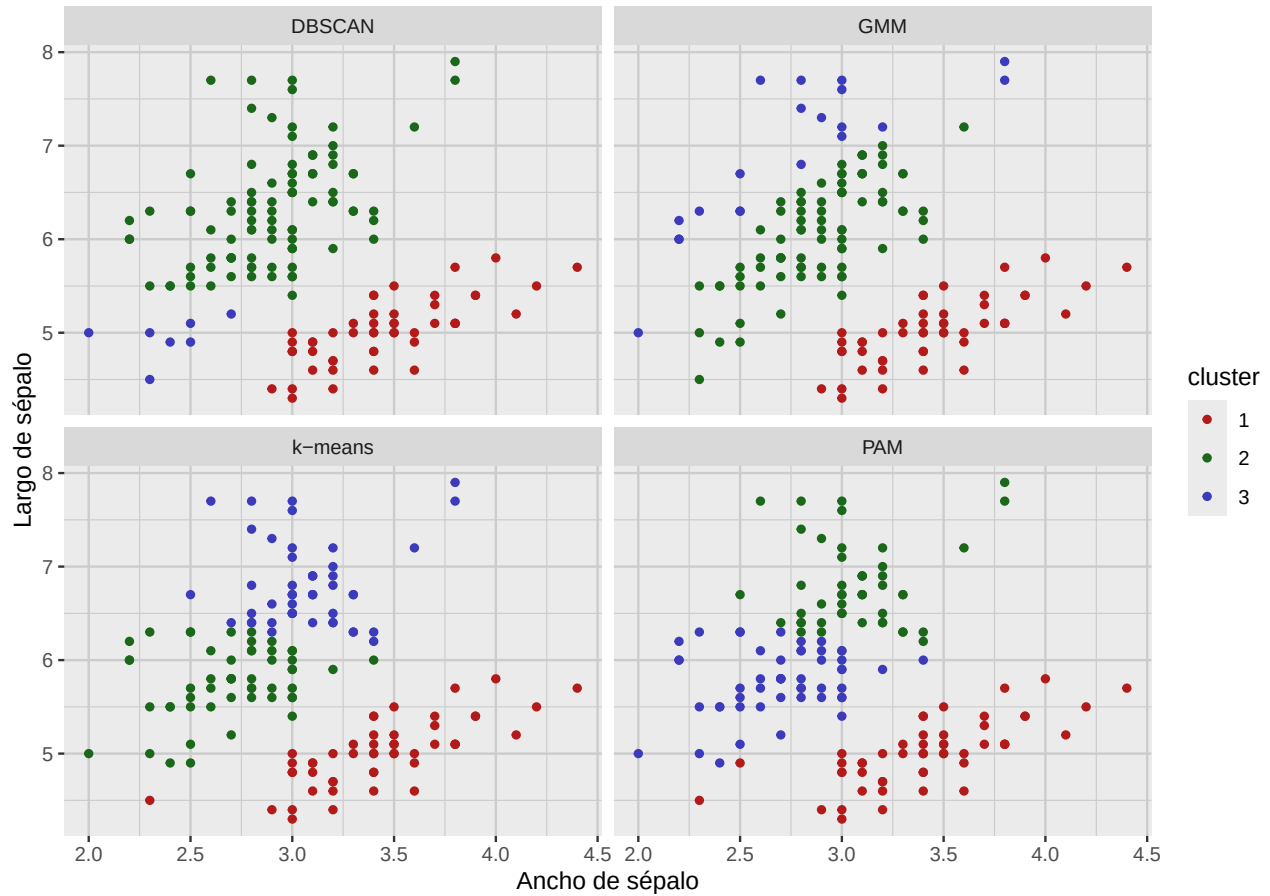


Figura 10.1: Resultados de agrupamiento en el conjunto iris usando únicamente Sepal.Width y Sepal.Length. Nótese la diferencia de formas de los conglomerados según el método.

